



The Open University

M263 Building Blocks of Software

Unit 9

Logic in practice

Study guide

The four sections of this unit are of similar length and importance. There are some practical computing activities associated with Sections 3 and 4. There is a software tool that you may find of help in association with Section 2, but use of this is optional. Overall, you should not spend more than about one tenth of your study time for this unit on practical computing work.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 6838 1

Contents

Introduction	4
1 Boolean expressions	4
1.1 Truth tables	5
1.2 Equivalent Boolean expressions	8
1.3 Disjunctive normal form	9
2 Parse trees	11
2.1 From parse tree to expression	12
2.2 Using a parse tree to find truth values	13
2.3 Valid Boolean expressions	15
2.4 From expression to parse tree	18
3 Using Boolean expressions	21
4 Logic and language	28
4.1 Two new operations	29
4.2 Composite propositions	31
4.3 Rewriting case statements	33
Summary of Unit 9	37
Solutions to the exercises	39
Index	47

Introduction

Most software systems involve selection in some form. If you were developing a system for booking holidays on the web, for example, then you would have to consider conditions such as: restrictions about the days of the week on which the holiday starts; conditions for clients organising their own travel; conditions for child reductions; and so on. These conditions will initially be expressed in natural language, as in the examples below.

If the start date is not a Saturday then price code C applies.

If the clients have children over five years old and are organising their own travel then they only get part of the child reduction.

In order to write corresponding software these conditions have to be expressed formally. In this unit and the next one you will develop skills to help you to move between natural language expressions and their formal counterparts. You will also develop techniques for rewriting formal expressions in ways that lead to code that is well-structured and consequently more straightforward to maintain. This unit is primarily concerned with developing formal logical ideas that help to achieve these aims.

A **Boolean expression** is an expression involving variables that represent values in *Bool*, and operations combining such values. In Section 1 we look at truth tables, which are a way of working with Boolean expressions. In particular, they show that different Boolean expressions may be **equivalent**, in that their values are always the same whatever the values of the variables in the expressions. Truth tables also lead us to a way of rewriting expressions in a standard format. In Section 2, **parse trees** are introduced as a technique for analysing expressions. We also discuss how to recognise a **well-formed formula**. In Section 3 we use the ideas of the first two sections to help with constructing code using conditional statements. Section 4 introduces two new Boolean operations and gives some initial practice in the important skill of moving between formal and natural language.

A well-formed formula is a Boolean expression that is syntactically correct. A fuller discussion is given on page 15.

1 Boolean expressions

Suppose that you are searching a library database for some music for a choir. The choir would like to sing either the Fauré Requiem or the Mozart Requiem and you wish to find the available editions. An advanced search engine might look like Figure 1.1. In the figure, the circles beside the operations AND, OR, NOT represent radio-buttons, and the dots indicate the button that has been selected from each choice. (A radio button is an interface component that allows a user to choose precisely one from a group of options; that is, only one radio button in a group can be selected at any one time.)

For example, searching in the Open University Library for a book either with both logic and metaphysics in the title or with author J.S.Mill produced just one result, namely:

Author: Parkinson, G. H. R.

Title: Logic and reality in Leibniz's metaphysics

Mozart	Composer
AND ☐	OR ☒
Fauré	Composer
AND ☒	OR ☐
Requiem	Piece

Figure 1.1

This type of search is called a Boolean search. The word Boolean is used because we are using the operations *AND*, *OR* and *NOT*. In this section we are going to look at the expressions of the type used in such searches.

1.1 Truth tables

A **Boolean variable** is a variable whose value lies in the set *Bool*; that is, a variable whose value is either **true** or **false**. Suppose that a, b and c are Boolean variables. In discussing Boolean expressions, we will call such variables **atoms**. We build up Boolean expressions by combining these atoms with Boolean operations. For example, $a \wedge (b \vee c)$ is built up from a, b and c using the operations $\vee$, read “or” and $\wedge$, read “and”. These were introduced in Block 1. There, we also introduced the function *NOT* on Boolean values. Here, we will use the notation $\neg$ for *NOT*, as this is the standard notation used in formal logic for “not”.

The meaning of the operations $\wedge, \vee$, and $\neg$ can be defined using tables that give the truth value of each of the expressions $a \wedge b$, $a \vee b$ and $\neg a$ for every possible combination of truth values for a and b . Such tables are called **truth tables**. We shall use T to denote **true** and F to denote **false**.

The table for $\wedge$ is:

a	b	$a \wedge b$
T	T	T
T	F	F
F	T	F
F	F	F

If a is **false** and b is **true** then $a \wedge b$ is **false**.

The table for $\vee$ is:

a	b	$a \vee b$
T	T	T
T	F	T
F	T	T
F	F	F

If a is **true** and b is **false** then $a \vee b$ is **true**.

The table for $\neg$ is:

a	$\neg a$
T	F
F	T

If a is **false** then $\neg a$ is **true**.

As discussed in Unit 1 Section 5, we can regard $\wedge, \vee$ and $\neg$ as functions. The truth tables above give, in effect, the postconditions of these functions.

We can use truth tables to set out the truth values of more complex Boolean expressions. Note the systematic arrangement of the rows in each truth table above. Having a standard pattern for such tables makes it easy to check that all possible combinations have been covered and also makes it easy to compare tables. The pattern for a Boolean expression involving three atoms is illustrated in Activity 1.1, below.

When dealing with an expression such as $a \wedge ((\neg b) \vee c)$ we need to determine the order in which to evaluate the operations. The main rule is to respect the bracketing. In this expression, we first evaluate $\neg b$, then combine the result with c using $\vee$, then combine the result of that with a using $\wedge$.

Activity 1.1

Give the truth table for $a \wedge ((\neg b) \vee c)$.

The mathematician George Boole (1815–1864) was born in Lincoln. He ran a school from 1835 until he was appointed to the chair of mathematics at Queen’s College, Cork in 1849. In 1854 he published “An investigation into the Laws of Thought, on Which are founded the Mathematical Theories of Logic and Probabilities.”

We shall do this regularly throughout the unit, to save space.

The comment on the right of each table gives an example of how the table is to be interpreted, for the row picked out by shading.

For an expression involving two atoms, the table has four (2×2) rows; for an expression involving three atoms, the truth table will have eight ($2 \times 2 \times 2$) rows, and so on.

In ordinary arithmetic we need to know the order of calculation in an expression such as $2 \times (4 - 3)$. The rule is to compute what is in the brackets first and then multiply by 2, giving the answer 2. If we had ignored the brackets and computed 2×4 first we would have obtained 5.

Discussion We start by drawing up a truth table with eight rows giving all the possible values of the atoms in the expression, which here are a, b and c . To the right of the columns giving the possible values of a, b and c , there is a column headed by the given Boolean expression $a \wedge ((\neg b) \vee c)$. We fill in truth values for this expression in stages, corresponding to the bracketing in the expression. The first step is to deal with $(\neg b)$. This simply involves changing the values in the column for b from T to F and from F to T respectively. We then deal with $(\neg b) \vee c$.

a	b	c	a	$\wedge$	$((\neg b) \vee c)$	c
T	T	T	T	F	T	T
T	T	F	T	F	F	F
T	F	T	T	T	T	T
T	F	F	T	T	T	F
F	T	T	F	F	T	T
F	T	F	F	F	F	F
F	F	T	F	T	T	T
F	F	F	F	T	T	F

The highlighted column gives the truth values of the expression $(\neg b) \vee c$.

These are obtained by combining the values in the two adjacent columns with $\vee$.

We use a systematic order for the rows, and will treat this order as standard. In the first four rows, the value of a is **true**, and in the last four rows a is **false**. The value of b is **true** in rows 1 and 2 and in rows 5 and 6, and is **false** in the other rows. The truth value of c alternates down the rows starting with **true**.

a	b	c	a	$\wedge$	$((\neg b) \vee c)$
T	T	T	T	T	T
T	T	F	T	F	F
T	F	T	T	T	T
T	F	F	T	T	T
F	T	T	F	F	T
F	T	F	F	F	F
F	F	T	F	F	T
F	F	F	F	F	T

The highlighted column now gives the truth values of the expression $a \wedge ((\neg b) \vee c)$.

These are obtained by combining the values in the two adjacent columns with $\wedge$.

a	b	c	$a \wedge ((\neg b) \vee c)$
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

This is the table that we want.

The last column gives the truth values of the given expression $a \wedge ((\neg b) \vee c)$.

We have simply removed extraneous columns from the table above.

In practice, there is no need to write out the table several times.

a	b	c	a	2	$\wedge$	$((\neg b) \vee c)$	1	$\vee$	c
T	T	T	T	T	F	T	T	T	T
T	T	F	T	F	F	F	F	F	F
T	F	T	T	T	T	T	T	T	T
T	F	F	T	T	T	T	T	F	F
F	T	T	F	F	F	T	T	T	T
F	T	F	F	F	F	F	F	F	F
F	F	T	F	F	T	T	T	T	T
F	F	F	F	F	T	T	T	F	F

First fill in the values for $a, \neg b$ and c . Then calculate the column labelled 1. Next calculate the column labelled 2 which gives the truth values of the whole expression.

You may find it helpful to cross out the working columns as you go along.

However you do your working, make sure that the column giving the answers is picked out clearly from the working columns.

One could calculate the entries in the truth table line-by-line, by substituting the appropriate value for each atom in the expression $a \wedge ((\neg b) \vee c)$. For example, the value for the fifth row has a **false**, b **true** and c **true**, and so is **false** $\wedge$ (**false** $\vee$ **true**), which is **false** $\wedge$ **true** = **false**.

But most people find the approach used above quicker and more reliable than working row by row.

Strategy 1.1

To draw up the truth table for a Boolean expression.

1. Determine the number of atoms in the expression and draw up a truth table with the left-hand columns headed by these atoms.
2. Fill these columns with T and F arranged systematically as in the examples above.
3. Write the full expression to the right of the variable headings with each term and each operation as a column heading. (Treat an expression of the form $(\neg a)$ as a single term.)
4. Use the brackets to determine the order of precedence of the operations and number them accordingly.
5. Calculate the values in each of these columns in turn.
6. When your calculation is finished, you may like to redraw the table showing the left-hand columns and the column giving the truth values of the expression.

Each different variable in the expression is an atom. For example, the expression $a \vee ((b \wedge (\neg a)) \vee b)$ contains two atoms.

To make it easy to determine the order in which operations should be performed, we have been placing brackets around terms of the form $\neg a$ and $\neg(a \vee b)$ whenever they have been part of a larger expression. However, too many brackets can make expressions hard to read. For this reason we will sometimes omit these brackets and have an underlying rule of precedence that says that terms of the form $\neg r$, where r is a Boolean expression, should be calculated as soon as the value of r has been found. So, for example, the expression $\neg a \vee b$ is interpreted as meaning $(\neg a) \vee b$.

We say that **negation has precedence**.

Activity 1.2

Give the truth tables for $\neg a \vee \neg b$ and $\neg(a \wedge b)$. What do you observe?

Discussion The first expression has truth table:

a	b	$\neg a$	1 $\vee$	$\neg b$
T	T	F	F	F
T	F	F	T	T
F	T	T	T	F
F	F	T	T	T

which gives

a	b	$\neg a \vee \neg b$
T	T	F
T	F	T
F	T	T
F	F	T

The second expression has truth table:

a	b	2 $\neg$	(a	1 $\wedge$	$b)$
T	T	F	T	T	T
T	F	T	T	F	F
F	T	T	F	F	T
F	F	T	F	F	F

which gives

a	b	$\neg(a \wedge b)$
T	T	F
T	F	T
F	T	T
F	F	T

Observe that the last columns of the two truth tables are the same. This shows that the truth values of the expressions $\neg a \vee \neg b$ and $\neg(a \wedge b)$ are the same for any combination of truth values of a and b .

We will use this method of displaying truth table calculations from now on. Shaded columns are calculated in the order of the numbers at the top.

Since the rows are given in the same order, we only need to look at the final column of each table to see this.

1.2 Equivalent Boolean expressions

In Activity 1.2, we found that the last columns of the two truth tables were the same. This shows that the truth values of the two expressions are the same for every combination of the truth values of the atoms in the expressions. Boolean expressions with this property are said to be **equivalent**. We write $r \equiv s$ when the expressions r and s are equivalent. We can use this idea, just as in algebra, to rearrange Boolean expressions into different forms. We will develop some tools to do this in Unit 10. For the moment, we will just see how to determine when two expressions are equivalent. Provided that there are not too many variables, this is most easily done by comparing the final columns of the truth tables of the expressions.

Recognising equivalent expressions is useful when comparing code and can also help to simplify code.

EXAMPLE 1.1

The expressions $a \wedge (b \vee c)$ and $(a \wedge b) \vee (a \wedge c)$ are equivalent. We write

$$a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c).$$

We see this by drawing up a truth table for each expression, and noting that the last columns of their truth tables are the same.

a	b	c	2 $\wedge$	1 $(b \vee c)$		a	b	c	$a \wedge (b \vee c)$
T	T	T	T	T		T	T	T	T
T	T	F	T	T		T	T	F	T
T	F	T	T	F		T	F	T	T
T	F	F	T	F		T	F	F	F
F	T	T	F	T		F	T	T	F
F	T	F	F	T		F	T	F	F
F	F	T	F	F		F	F	T	F
F	F	F	F	F		F	F	F	F

which gives

a	b	c	$a \wedge (b \vee c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	F
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

a	b	c	1 $(a \wedge b)$	3 $\vee$	2 $(a \wedge c)$		a	b	c	$(a \wedge b) \vee (a \wedge c)$
T	T	T	T	T	T		T	T	T	T
T	T	F	T	T	F		T	T	F	T
T	F	T	T	F	T		T	F	T	T
T	F	F	T	F	F		T	F	F	F
F	T	T	F	T	F		F	T	T	F
F	T	F	F	T	F		F	T	F	F
F	F	T	F	F	F		F	F	T	F
F	F	F	F	F	F		F	F	F	F

which gives

a	b	c	$(a \wedge b) \vee (a \wedge c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	F
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

Using the approach in Example 1.1, we can show that $(a \vee b) \vee c \equiv a \vee (b \vee c)$, and that $(a \wedge b) \wedge c \equiv a \wedge (b \wedge c)$. These equivalences mean that we can write expressions such as $a \wedge b \wedge c$ or $a \vee b \vee c$ without brackets, with no danger of confusion.

Activity 1.3

Show that $\neg(a \vee b) \equiv \neg a \wedge \neg b$.

Discussion We draw up truth tables for the two expressions.

$a \quad b$		2	$\neg$		1	$(a \vee b)$	
T	T	F	T	T	T	T	
T	F	F	T	T	F	F	
F	T	F	F	T	T	T	
F	F	T	F	F	F	F	

which gives

a	b	$\neg(a \vee b)$
T	T	F
T	F	F
F	T	F
F	F	T

$a \quad b$		$\neg a$	1	$\wedge$	$\neg b$
T	T	F	F	F	F
T	F	F	F	T	T
F	T	T	F	F	F
F	F	T	T	T	T

which gives

a	b	$\neg a \wedge \neg b$
T	T	F
T	F	F
F	T	F
F	F	T

Since the last columns of the two truth tables are the same, the two expressions are equivalent.

1.3 Disjunctive normal form

One technique for handling complex expressions is to write them in what is called a **normal form**. We will look at one example of such a normal form, known as the **disjunctive normal form**. This is based on the rows of the truth table. The disjunctive normal form of an expression contains one term for each row of its truth table that evaluates to **true**. For example, the truth table for an expression in two Boolean values has four rows. Now each of the expressions $a \wedge b$, $a \wedge \neg b$, $\neg a \wedge b$ and $\neg a \wedge \neg b$ has a truth table in which precisely one of the four rows evaluates to **true**.

a	b	$a \wedge b$
T	T	T
T	F	F
F	T	F
F	F	F

a	b	$a \wedge \neg b$
T	T	F
T	F	T
F	T	F
F	F	F

a	b	$\neg a \wedge b$
T	T	F
T	F	F
F	T	T
F	F	F

a	b	$\neg a \wedge \neg b$
T	T	F
T	F	F
F	T	F
F	F	T

We can form an expression corresponding to any choice of rows in the truth table evaluating to T, by combining an appropriate selection of these terms with $\vee$.

For example, the truth table for the expression $\neg(a \wedge b)$ was given in Activity 1.2. In that, note that the last three rows of the truth table evaluate to **true**, and the first row to **false**. Now the expression

$$(a \wedge \neg b) \vee (\neg a \wedge b) \vee (\neg a \wedge \neg b)$$

will have a truth table with the same entries in the final column, and so is equivalent to the original expression. We have

$$\neg(a \wedge b) \equiv (a \wedge \neg b) \vee (\neg a \wedge b) \vee (\neg a \wedge \neg b).$$

The expression $(a \wedge \neg b) \vee (\neg a \wedge b) \vee (\neg a \wedge \neg b)$ is the **disjunctive normal form** of the expression $\neg(a \wedge b)$. For any Boolean expression involving two atoms, we can find the disjunctive normal form in a similar way.

We can extend this idea to Boolean expressions involving more than two atoms. The first step is to associate each row of the truth table with a term. For an expression with three atoms, these terms are tabulated overleaf. In each case the term is an expression whose truth table has **true** for only one row — that to which the term corresponds.

This approach can be extended in a natural way to Boolean expressions involving more than three atoms. With four atoms, there are sixteen different basic terms that may appear in the disjunctive normal form, and so on.

a	b	c	Associated term
T	T	T	$a \wedge b \wedge c$
T	T	F	$a \wedge b \wedge \neg c$
T	F	T	$a \wedge \neg b \wedge c$
T	F	F	$a \wedge \neg b \wedge \neg c$
F	T	T	$\neg a \wedge b \wedge c$
F	T	F	$\neg a \wedge b \wedge \neg c$
F	F	T	$\neg a \wedge \neg b \wedge c$
F	F	F	$\neg a \wedge \neg b \wedge \neg c$

The **disjunctive normal form** of a Boolean expression is formed as follows.

Find its truth table.

Write down the term corresponding to each row evaluating to **true**.

Connect these terms by $\vee$.

The disjunctive normal form is always equivalent to the original expression.

The Boolean operation $\vee$ is sometimes known as **disjunction**, which is why this is known as the disjunctive normal form. Some texts refer to the disjunctive normal form as the **complete sum-of-products form**.

There is a different normal form where terms such as $a \vee \neg b \vee c$ are connected by $\wedge$. This is known as the conjunctive normal form.

Activity 1.4

Find the disjunctive normal form of each of the following expressions.

- (a) $a \vee b$.
- (b) $\neg a \vee (\neg b \wedge c)$.
- (c) $a \wedge (b \vee c)$. (Look back to Example 1.1.)

Discussion

- (a) First find the truth table for $a \vee b$.

a	b	$a \vee b$	Term
T	T	T	$a \wedge b$
T	F	T	$a \wedge \neg b$
F	T	T	$\neg a \wedge b$
F	F	F	$\neg a \wedge \neg b$

For each row evaluating to **true**, write down the corresponding term (these have been shown above next to the truth table). Connect these by $\vee$ to get the disjunctive normal form for $a \vee b$, which is:

$$(a \wedge b) \vee (a \wedge \neg b) \vee (\neg a \wedge b).$$

- (b) Find the truth table and pick out the rows evaluating to **true**.

a	b	c	$\neg a$	$\frac{2}{\vee}$	$(\neg b$	$\frac{1}{\wedge}$	$c)$
T	T	T	F	F	F	F	T
T	T	F	F	F	F	F	F
T	F	T	F	T	T	T	T
T	F	F	F	F	T	F	F
F	T	T	T	T	F	F	T
F	T	F	T	T	F	F	F
F	F	T	T	T	T	T	T
F	F	F	T	T	T	F	F

which gives

a	b	c	$\neg a \vee (\neg b \wedge c)$
T	T	T	F
T	T	F	F
T	F	T	T
T	F	F	F
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	T

Hence the disjunctive normal form of $\neg a \vee (\neg b \wedge c)$ is

$$(a \wedge \neg b \wedge c) \vee (\neg a \wedge b \wedge c) \vee (\neg a \wedge b \wedge \neg c) \vee (\neg a \wedge \neg b \wedge c) \vee (\neg a \wedge \neg b \wedge \neg c).$$

- (c) The truth table for $a \wedge (b \vee c)$ was found in Example 1.1, so we do not need to repeat it. In that truth table, the first three rows evaluated to **true**. Combining the three corresponding terms with $\vee$ gives the disjunctive normal form of $a \wedge (b \vee c)$, which is:

$$(a \wedge b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge \neg b \wedge c).$$

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Construct a truth table for a Boolean expression.
- ☐ Use truth tables to test the equivalence of two Boolean expressions.
- ☐ Find the disjunctive normal form of a Boolean expression.

Exercises on Section 1

Exercise 1.1

- (a) Draw up truth tables for $(a \vee b) \vee c$ and $a \vee (b \vee c)$, and confirm that the last columns are the same.
- (b) Draw up truth tables for $(a \wedge b) \vee c$ and $a \wedge (b \vee c)$ and observe that the last columns are different.

This shows that brackets are essential in these expressions.

Exercise 1.2

Show that the expressions $a \vee (b \wedge c)$ and $(a \vee b) \wedge (a \vee c)$ are equivalent.

Exercise 1.3

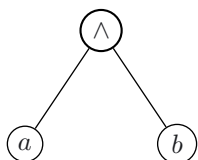
Find the disjunctive normal form for each of the expressions:

- (a) $a \wedge (b \vee \neg a)$; (b) $a \vee (\neg b \wedge c)$.

2 Parse trees

There is a graphical approach to analysing formulae such as Boolean expressions that you may find helpful. The approach is based on a form of tree, which we call a **parse tree**. In a parse tree, each leaf node is an atom. Each non-leaf node is an operation, and the branches from such nodes lead either to an atom or to a subtree that is again a valid parse tree. If the operation is $\vee$ or $\wedge$, the node has two branches, while if it is $\neg$, the node has one branch.

As a simple example, we give the parse tree for the expression $a \wedge b$. This has $\wedge$ at the root node, and two branches leading to leaf nodes containing the atoms a and b respectively.



A parse tree is *not* a binary tree, but we can still use some of the terminology associated with trees introduced in Unit 8.

A leaf node is a node that has no branches below it. The root node is the node at the top of the tree.

2.1 From parse tree to expression

We will first look at how we interpret a parse tree; that is, how we find the expression that it represents. There are two ways we can do this, working either from the bottom up or from the top down.

The following two examples describe these two approaches for the same tree, that in Figure 2.1.

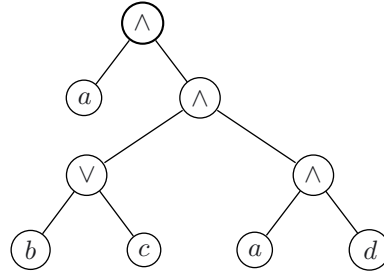


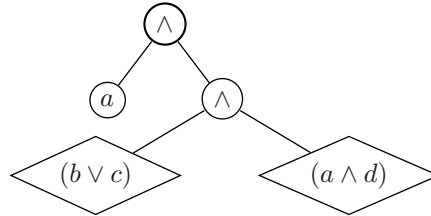
Figure 2.1

EXAMPLE 2.1

We can find the expression represented by the parse tree in Figure 2.1 working “bottom up” as follows. Start at the lowest level of the tree, and look at the nodes whose branches lead to leaf nodes. Replace each node at this level by the expression that it represents.

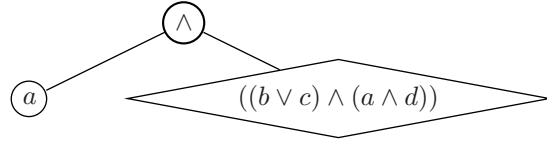
Step 1

Where an expression has been substituted for a subtree, we have placed the expression in brackets, so that it is treated as a single item at the next stage. (We have also changed the shape of the nodes.)



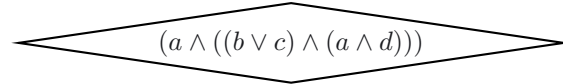
Step 2

Proceed up the levels of the tree, replacing operations by expressions as required.



Step 3

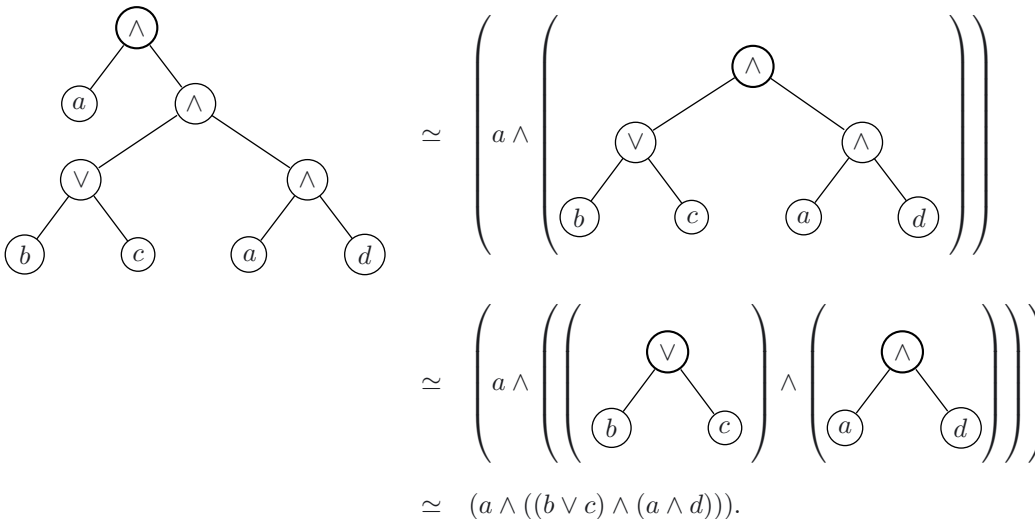
The final step is to replace the root node.



We can now see that the original tree represents the expression $(a \wedge ((b \vee c) \wedge (a \wedge d)))$.

EXAMPLE 2.2

Using the top down approach we start from the root node, as below.



We write the tree as an expression where the arguments of the operation at the root node are written as subtrees if necessary. In the tree in Figure 2.1, the root node has the operation $\wedge$. The first argument for this is the atom a . The second argument remains to be evaluated, so is left as a tree in the expression. We use the notation $\simeq$ between the expressions. Read this as “can be written as”. The large brackets round the trees represent brackets in the final expression being found. As each node is dealt with, brackets are introduced as necessary.

These two examples have shown two different approaches to finding the expression represented by a parse tree. There is no preference as to which approach is used; use whichever you find easier to follow.

Where a parse tree has a $\neg$ at a node, the branch below that node is shown going straight down. It is neither a left branch or a right branch. (Consequently, as noted earlier, a parse tree is not a binary tree.)

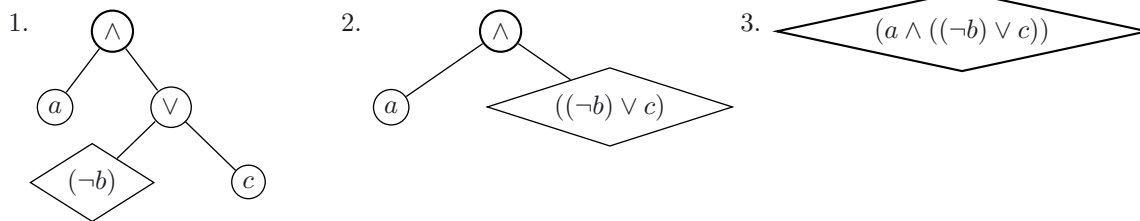
Activity 2.1

Find the expression represented by the parse tree in Figure 2.2

Discussion We will show both approaches.

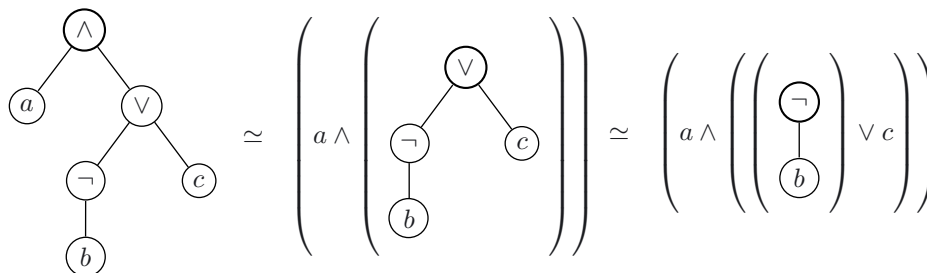
Working from the bottom up

The steps are as shown below.



From this we can see the expression represented is $(a \wedge ((\neg b) \vee c))$.

Working from the top down



This expression can equally well be written as $a \wedge ((\neg b) \vee c)$, or as $a \wedge (\neg b \vee c)$, using the rule giving precedence to negation.

Again, we see that the expression represented by the tree is $(a \wedge ((\neg b) \vee c))$.

2.2 Using a parse tree to find truth values

We can use a parse tree to find the truth value of the expression it represents, for given truth values of the atoms in the expression. To do this, put the truth values for the atoms on the branches leading from them, and then work up the tree.

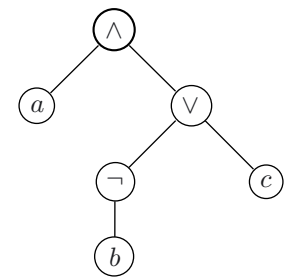


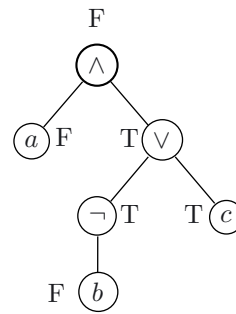
Figure 2.2

EXAMPLE 2.3

The parse tree on the right represents $a \wedge (\neg b \vee c)$ (see Activity 2.1).

The parse tree is annotated with truth values, which show the following.

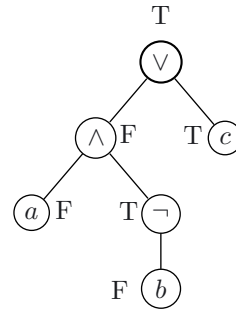
If a and b are **false** and c is **true**, then the truth value of $a \wedge (\neg b \vee c)$ is **false**.



The parse tree on the right represents $(a \wedge \neg b) \vee c$.

The annotation of the tree shows the following.

If a and b are **false** and c is **true**, then the truth value of $(a \wedge \neg b) \vee c$ is **true**.

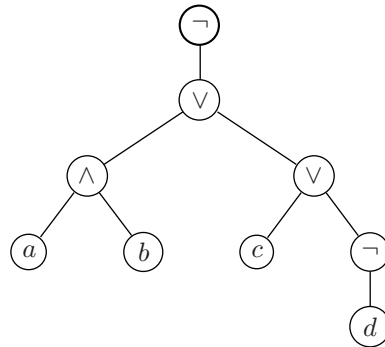


Note the extent of the difference in these two trees considering that the only difference in the expressions is in the positioning of brackets.

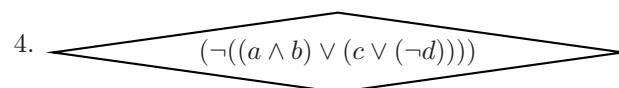
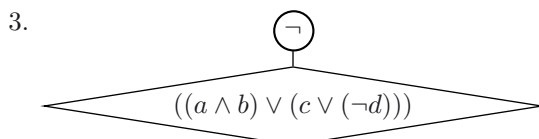
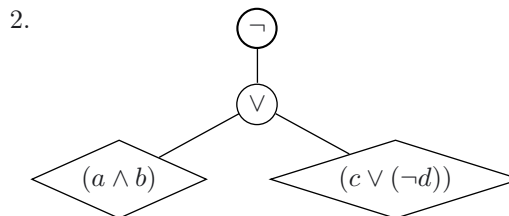
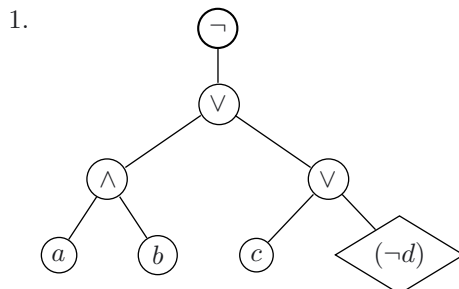
Activity 2.2

What Boolean expression is represented by the parse tree on the right?

Find the truth value of the Boolean expression when a and c are **true**, and b and d are **false**.



Discussion We will work from the bottom up.



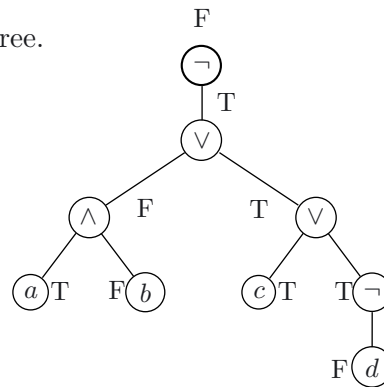
Thus the expression represented by the tree is $\neg((a \wedge b) \vee (c \vee (\neg d)))$.

To find the truth value we annotate the tree.

First enter the truth values for the atoms: T for a ; F for b ; T for c and F for d .

We see that, for these truth values of the atoms, the expression

$(\neg((a \wedge b) \vee (c \vee (\neg d))))$ is **false**.



2.3 Valid Boolean expressions

We need to be aware that Boolean expressions have a syntax. That is, the way atoms and operations are combined must obey certain rules if the resulting expression is to make sense. For example, the expression $a \neg b$ does not make sense. This expression does not obey the rules!

Some texts use the phrase **well-formed formula** to describe a properly constructed expression. We will call a Boolean expression a well-formed formula if it is built up according to the rules given below. Any formula built up using these rules is certain to be syntactically correct.

- atom** Atoms, such as $a, b, c, \dots$ are well-formed formulae.
- negation** If s is a well-formed formula then $(\neg s)$ is well-formed.
- conjunction** If s, t are well-formed formulae then $(s \wedge t)$ is well-formed.
- disjunction** If s, t are well-formed formulae then $(s \vee t)$ is well-formed.

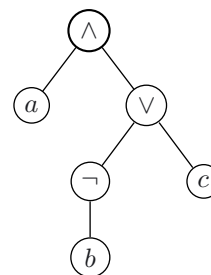
There is a close relationship between these rules and parse trees. The procedure we described in Subsection 2.1 for finding the Boolean expression corresponding to a parse tree will always result in a well-formed formula. Consequently we can use the parse tree for an expression to show that it has correct syntax.

An important check for program compilers to make is that any formulae used are syntactically correct.

In Section 4 we shall introduce two further operations, $\Rightarrow$ and $\Leftrightarrow$. The definition of a well-formed formula can be extended to include these.

EXAMPLE 2.4

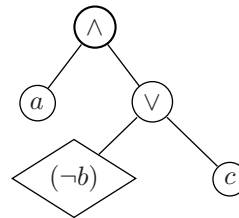
We saw in Activity 2.1 that the expression $(a \wedge ((\neg b) \vee c))$ has the parse tree shown on the right.



Think about the bottom up approach to determining the expression from the parse tree. The trees given below and on the right go through that process. On the left below we show how each step in the process corresponds to one of the rules for creating a well-formed formula.

b is well formed (atom).

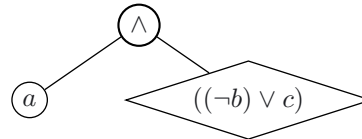
So $(\neg b)$ is well formed (negation). (A)



$(\neg b)$ is well formed — see (A).

c is well formed (atom).

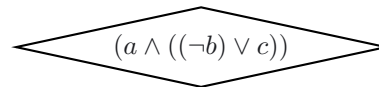
So $((\neg b) \vee c)$ is well formed (disjunction). (B)



a is well-formed (atom).

$((\neg b) \vee c)$ is well-formed — see (B).

So $(a \wedge ((\neg b) \vee c))$ is well-formed (conjunction).



EXAMPLE 2.5

We can present in tabular form the process of building up a well-formed formula, without direct reference to the tree. This is done below for the previous example. In the table, we give a reference number for each well-formed formula that we form “along the way”. These numbers are in the column on the left below.

Number	Formula	How formed
1	b	atom
2	$(\neg b)$	negation(1)
3	c	atom
4	$((\neg b) \vee c)$	disjunction(2, 3)
5	a	atom
6	$(a \wedge ((\neg b) \vee c))$	conjunction(5, 4)

We will use the term **parse table** to describe this process.

Activity 2.3

Look back to Activity 2.2, where the parse tree for the expression $\neg((a \wedge b) \vee (c \vee (\neg d)))$ was given. Draw up a parse table for this expression.

You may find it helpful to refer to the steps given in the comment on Activity 2.2.

Discussion A parse table is given below. (You may have some of the lines in a different order.)

Number	Formula	How formed
1	d	atom
2	$(\neg d)$	negation(1)
3	a	atom
4	b	atom
5	$(a \wedge b)$	conjunction(3, 4)
6	c	atom
7	$(c \vee (\neg d))$	disjunction(6, 2)
8	$((a \wedge b) \vee (c \vee (\neg d)))$	disjunction(5, 7)
9	$\neg((a \wedge b) \vee (c \vee (\neg d)))$	negation(8)

Note that drawing up a parse table in this way shows that the given expression is a well-formed formula.

EXAMPLE 2.6

The expression $(a \wedge (\neg b) \vee c)$ is not a well-formed formula. We can see this as follows. Each of the rules given for creating a well-formed formula adds a pair of brackets for each operation symbol. So a well-formed formula will have equal numbers of open brackets and of close brackets, and there will be the same number of each as there are operation symbols ($\wedge$, $\vee$ or $\neg$) in the expression. This expression has three operation symbols and only two pairs of brackets — so it does not have enough brackets.

We will not be concerned with showing formally that formulae are not well-formed at this stage.

At the start of this subsection we gave rules for creating a well-formed formula. These rules insert enough brackets into the expression to be sure that it can be interpreted. However, we can interpret expressions with rather fewer brackets in them. For example, we can remove the outer brackets from a formula and it can still be interpreted. Also, if we give precedence to negation in the way indicated in Section 1, we can interpret an expression such as $\neg a \vee b$ as meaning the same as the well-formed formula $((\neg a) \vee b)$. Since an excess of brackets can make expressions harder for a person to read, we sometimes give expressions with fewer brackets than would appear in a well-formed formula. There is a unique parse tree corresponding to a syntactically correct expression such as $a \wedge (\neg b \vee c)$. That is because there is a unique way of putting back the brackets that have been omitted. The corresponding well-formed formula is $(a \wedge ((\neg b) \vee c))$.

But not for a machine!

There is another way in which we may omit brackets. We can interpret an expression such as $a \wedge b \wedge c$. This is because of the equivalence

$$((a \wedge b) \wedge c) \equiv (a \wedge (b \wedge c)).$$

We used this when writing the disjunctive normal form of an expression.

There are two ways of adding brackets to the expression $a \wedge b \wedge c$ to make it into a well-formed formula, but since these lead to equivalent expressions we can interpret the expression $a \wedge b \wedge c$ given without brackets. Now the expressions $((a \wedge b) \wedge c)$ and $(a \wedge (b \wedge c))$ have different parse trees. So use of expressions such as $a \wedge b \wedge c$ will confuse the relationship between expressions and parse trees. We now look at how we can find the parse tree corresponding to a valid expression. In doing this, we shall always include enough brackets to ensure that there is a unique way of adding brackets to the expression to give a well-formed formula, so that there is a unique parse tree for the expression.

We note, in passing, that the expression in Example 2.6 is not only not well-formed, it cannot be interpreted at all (and so is not syntactically correct). This is because there is no equivalence between the expressions $(x \wedge (y \vee z))$ and $((x \wedge y) \vee z)$, so there are different ways of adding brackets to that expression which lead to genuinely different (i.e., non-equivalent) well-formed formulae.

Activity 2.4

The expression $\neg c \wedge (\neg(a \vee b) \wedge c)$ is syntactically correct, but to interpret it we need to use the precedence rule for negation. Add brackets to make it into a well-formed formula.

Discussion The brackets around each negation have been omitted, as have the outer brackets around the whole expression. Adding these gives the well-formed formula:

$$((\neg c) \wedge ((\neg(a \vee b)) \wedge c)).$$

As a quick check, note that this expression has five each of: open brackets, close brackets, and operation symbols.

Counting brackets is a quick check that may reveal errors. It is not a proof that the expression is well-formed.

2.4 From expression to parse tree

Given an expression, the key to drawing its parse tree is to be able to recognise the root node. Finding this will identify either one or two subexpressions (whose root nodes can then be determined). We can determine the root node by counting brackets, and recording the count as each operation is reached, as laid out below. For the procedure to work, we need first to make sure that the expression has the appropriate brackets. We want the expression to be a well-formed formula with the outer pair of brackets removed.

- Insert brackets around expressions of the form $\neg r$, wherever they have been omitted.
- Remove any outer brackets.
- Start with count 0. Work through the expression from left to right. Add 1 to your count for each opening bracket, and subtract 1 for each closing bracket.
- Note the count when you pass an operation symbol (either $\wedge$, $\vee$ or $\neg$).
- The root node will be the operation where the bracket count is zero.

By a **subexpression** we mean a part of an expression that can itself be represented as a parse tree.

Each $\neg$ should have an opening bracket directly before it in the expression.

Activity 2.5

Find the root node for each of the following expressions.

- $(a \vee b) \wedge (\neg(b \wedge c)).$
- $((a \wedge b) \vee (c \vee d)).$
- $((a \vee (b \wedge \neg c)) \wedge (\neg(c \vee d))).$
- $(a \wedge (\neg(b \vee (\neg(c \wedge d)))))$.

Discussion We use the bracket counts. In each case, the bracket count is 0 for just one operation symbol, which gives the root node.

Note in each case that the bracket count has returned to 0 when we reach the end of the expression. This provides a useful check.

- This expression already has the appropriate bracketing.

$$\begin{array}{ccccccc} (a & \vee & b) & \wedge & (\neg & (b & \wedge & c)) \\ 1 & & & 0 & 1 & & 2 \end{array}$$

- The expression has outer brackets. Remove these, to get $(a \wedge b) \vee (c \vee d)$.

$$\begin{array}{ccccccc} (a & \wedge & b) & \vee & (c & \vee & d) \\ 1 & & & 0 & & & 1 \end{array}$$

- We need to insert brackets around $\neg c$. Also, there are outer brackets, which we remove. This gives $(a \vee (b \wedge (\neg c))) \wedge (\neg(c \vee d))$.

$$\begin{array}{ccccccccccc} (a & \vee & (b & \wedge & (\neg & c))) & \wedge & (\neg & (c & \vee & d)) \\ 1 & & 2 & 3 & & 0 & 1 & & 2 \end{array}$$

- We need to remove the outer brackets, to get $a \wedge (\neg(b \vee (\neg(c \wedge d))))$.

This will always be the case for a well-formed formula with the outer brackets removed.

$$\begin{array}{ccccccc}
 a & \wedge & (& \neg & (b & \vee & (& \neg & (c & \wedge & d))) \\
 0 & & 1 & & 2 & & 3 & & 4 & &
 \end{array}$$

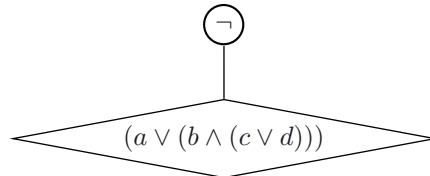
We can use this idea to draw a parse tree for a given expression.

EXAMPLE 2.7

We will draw the parse tree of $(\neg(a \vee (b \wedge (c \vee d))))$. The bracket count is:

$$\begin{array}{ccccccc}
 \neg & (& a & \vee & (& b & \wedge & (& c & \vee & d)) \\
 0 & & 1 & & 2 & & 3 & & 4 & &
 \end{array}$$

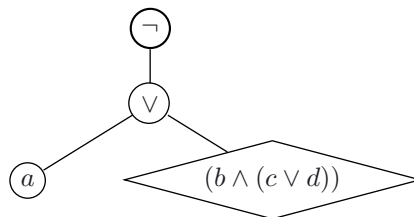
Thus the operation at the beginning, $\neg$, must be the root node. Hence our tree starts as shown on the right.



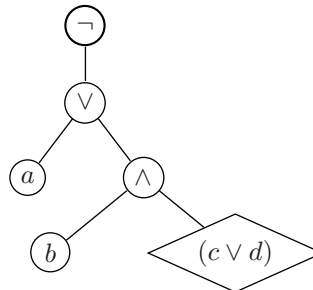
To perform the bracket count for the subexpression $(a \vee (b \wedge (c \vee d)))$, first remove the outer brackets.

$$\begin{array}{ccccccc}
 a & \vee & (& b & \wedge & (& c & \vee & d)) \\
 0 & & 1 & & 2 & & 3 & & 4 & &
 \end{array}$$

The next root node is the operation $\vee$ at count 0. This leads to the two subexpressions a and $(b \wedge (c \vee d))$, giving the tree on the right.

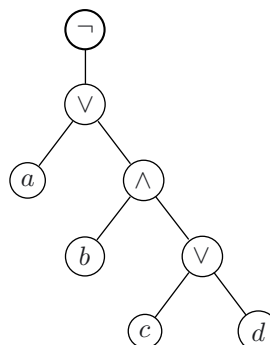


The subexpression a is an atom. In the subexpression $(b \wedge (c \vee d))$, the root node is $\wedge$, with subexpressions b and $(c \vee d)$.



We will no longer show bracket counts explicitly — they need only be used when finding the root node is tricky.

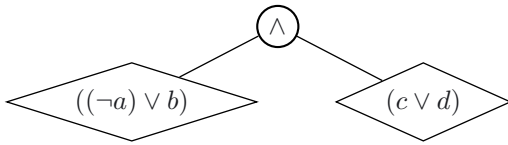
Finally the root node in $(c \vee d)$ is $\vee$, with subexpressions c and d .



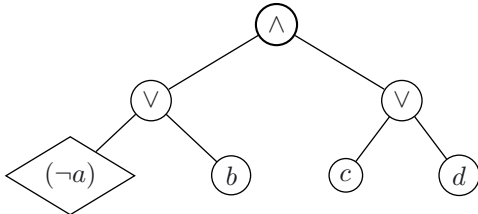
Activity 2.6

Draw the parse tree of $((\neg a) \vee b) \wedge (c \vee d)$. What is the value of the expression when a , c and d are **true** and b is **false**?

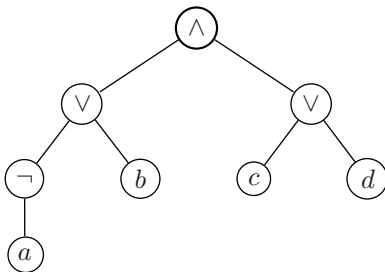
Discussion The root node is $\wedge$ with two subexpressions $((\neg a) \vee b)$ and $(c \vee d)$.



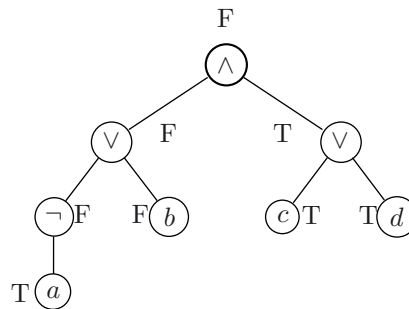
The first of these has $\vee$ as root with subexpressions $(\neg a)$ and b . The second also has $\vee$ as a root node and has subexpressions c and d .



Finally we replace $(\neg a)$ by a root node $\neg$, and one subexpression a .



As shown on the right, we can use the parse tree to obtain the truth value of the expression for the given truth values of the atoms. We see that, for these values of the atoms, the expression is **false**.



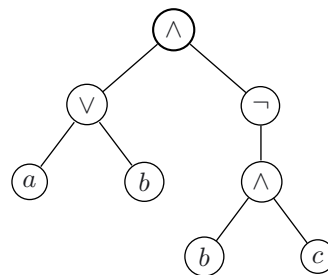
Activity 2.7

For the expression $((a \vee b) \wedge (\neg(b \wedge c)))$, give a parse tree and a parse table.

Discussion

The parse table and parse tree for the expression are given below.

Number	Formula	How formed
1	b	atom
2	c	atom
3	$(b \wedge c)$	conjunction(1, 2)
4	a	atom
5	$(a \vee b)$	disjunction(4, 1)
6	$\neg(b \wedge c)$	negation(3)
7	$((a \vee b) \wedge (\neg(b \wedge c)))$	conjunction(5, 6)



Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Determine the expression represented by a parse tree.
- ☐ Use a parse tree to determine the truth or falsity of an expression for given values of the atoms.
- ☐ Construct a parse tree for a Boolean expression.
- ☐ Give a parse table to verify that a Boolean expression is valid.

Exercises on Section 2

Exercise 2.1

What Boolean expression does the parse tree in Figure 2.3 represent? Find the truth value of the expression when a , b and d are **false** and c is **true**.

Exercise 2.2

For the expression $((a \wedge \neg b) \vee (\neg c \vee d))$, draw its parse tree. Find the truth value of the expression when b and c are **true** and a and d are **false**.

Exercise 2.3

For the expression $(a \wedge (\neg b)) \vee (\neg c)$, give a parse tree and construct a parse table. Is this expression a well-formed formula?

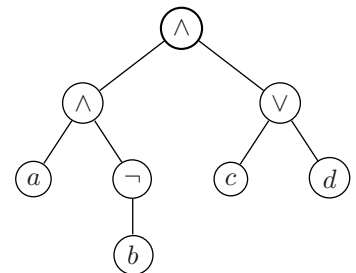


Figure 2.3

3 Using Boolean expressions

Computer code may involve conditional statements, where the desired outcome depends on the current values of one or more Boolean variables. Such conditional constructions can become elaborate, if the outcome depends on several values in a complicated way. Here, we shall look at ways of organising conditional statements. In the code fragments that we shall consider, the effect of the code will be to set the value of a single variable. This is to focus our discussion on the structure of the conditional statement. (In practice, these conditional statements would usually be part of a larger piece of code. The desired effect is then likely to be execution of a block of code appropriate to each condition.)

At the design stage, the conditions determining the desired outcome may relate to the outside world. For example, suppose that a company has a policy about offering mortgages that depends on the following.

- r : The borrower has sufficient income.
- h : The house has sufficient value (according to a survey).
- p : The borrower has no other property.

Each of these is an assertion (about conditions in the outside world) that may be either **true** or **false**. We call such an assertion a **proposition**. In code, such propositions will be represented by variables of type **Bool**.

Suppose that you have been given the code below (without documentation!). This code controls when a mortgage should be offered.

```

/* Fragment 1 */
var offer in mString
if (NOT(r) /\ (NOT(h) /\ p)) then
    {offer.setString("do not make")}    //(1)
else
{
    if (h) then
        {offer.setString("make")}      //(2)
    else
        {offer.setString("do not make")}    //(3)
}

```

The effect of this fragment is to assign a value to the variable *offer*. We will use this device in this section and the next to represent what in practice would probably be execution of an appropriate block of code in each case.

It is not that easy to understand from this code precisely when a mortgage should be offered. We can use a truth table to assist in analysing when each possible outcome will occur. This will enable us to design equivalent code that is more clearly structured, and so easier to understand and maintain. It is then also easier to check that the code does what is intended.

Activity 3.1

Give truth tables for appropriate Boolean expressions that show when each of statements (1), (2) and (3) in Fragment 1 above will be executed. Hence determine the condition under which the outcome is that *offer* is set to “make”. Write a simpler code fragment with the same effect.

Discussion As in Sections 1 and 2, we replace *NOT* by $\neg$. The first **if** condition is $\neg r \vee (\neg h \wedge p)$, and (1) is executed if this is **true**. If this is **false**, then the first **else** is entered. So (2) is executed if $\neg r \vee (\neg h \wedge p)$ is **false** and *h* is **true**, while (3) is executed if $\neg r \vee (\neg h \wedge p)$ is **false** and *h* is also **false**.

In Activity 1.4(b), we found the truth table for an expression with the same structure as $\neg r \vee (\neg h \wedge p)$ (but with different identifiers for the atoms). We can use this as a starting point, and go from there to find the truth tables for expressions corresponding to statements (2) and (3). To save space, we give the truth tables for all three expressions side-by-side.

<i>r</i>	<i>h</i>	<i>p</i>	$\neg r \vee (\neg h \wedge p)$	$\neg(\neg r \vee (\neg h \wedge p))$	$\wedge$	<i>h</i>	$\neg(\neg r \vee (\neg h \wedge p))$	$\wedge$	$\neg h$
T	T	T	F	T	T	T	T	F	F
T	T	F	F	T	T	T	T	F	F
T	F	T	T	F	F	F	F	F	T
T	F	F	F	T	F	F	T	T	T
F	T	T	T	F	F	T	F	F	F
F	T	F	T	F	F	T	F	F	F
F	F	T	T	F	F	F	F	F	T
F	F	F	T	F	F	F	F	F	T
			(1)		(2)		(3)		

Column (2) corresponds to Statement (2), which is the only situation where *offer* is set to “make”. This Boolean expression has disjunctive normal form $(r \wedge h \wedge p) \vee (r \wedge h \wedge \neg p)$. This is equivalent to $r \wedge h$. (In the truth table, column (2) contains T in exactly the same rows as those where $r \wedge h$ is **true**.)

We now see that the value of p is not relevant. If the borrower has sufficient income and the house has sufficient value, then a mortgage should be offered regardless of whether the borrower has another property. This was not clear from the original code. The fragment below has the same effect.

```
var offer in mString
if (r /\ h) then
  {offer.setString("make")}
else
  {offer.setString("do not make")}
```

The truth table in the discussion of Activity 3.1 enables us to see the outcome of the code in Fragment 1 for any possible combination of truth values for the variables r , h and p . The possible outcomes are: “make” (a mortgage offer should be made), or “do not make” (a mortgage offer should not be made). We can tabulate the outcomes as below. We refer to this as an **outcome table**.

r	h	p	outcome
T	T	T	“make”
T	T	F	“make”
T	F	T	“do not make”
T	F	F	“do not make”
F	T	T	“do not make”
F	T	F	“do not make”
F	F	T	“do not make”
F	F	F	“do not make”

In Activity 3.1, we were able to simplify some given code. A more direct approach is first to analyse the outcomes that are wanted, then to code a conditional statement. We will discuss two ways of drawing up the desired outcomes. One is to give an outcome table. An alternative is to give a tree showing the outcomes. Such an **outcome tree** will again show the desired outcome for each possible combination of truth values of the variables. Both approaches are illustrated in the next example.

EXAMPLE 3.1

A supermarket sends sets of vouchers to their storecard holders each month. It likes to use its records to target the vouchers. In a particular month it has two sets of vouchers, one set for detergents and one set for organic lines. Customers who have spent more than 200 pounds in the previous month should receive both sets of vouchers. Customers who bought any form of washing powder will be sent the detergent vouchers and customers who bought any organic products will be sent the organic vouchers.

The three propositions whose values will control the outcome are these.

m : The customer spent more than £200 last month.

g : The customer bought some organic products last month.

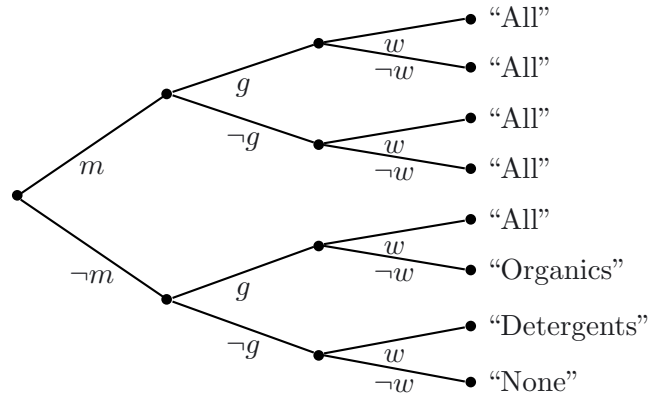
w : The customer bought washing powder last month.

The possible outcomes are these.

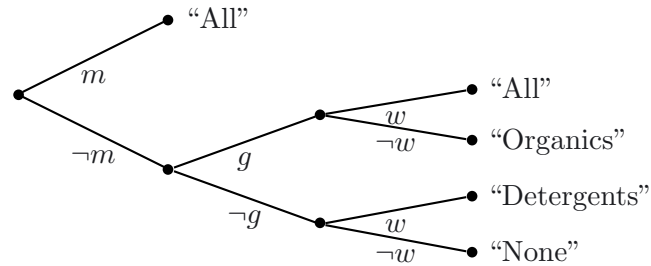
“All”	The customer will be sent both sets of vouchers.
“Detergents”	The customer will be sent the vouchers for detergents.
“Organics”	The customer will be sent the vouchers for organic products.
“None”	The customer will be sent no vouchers.

The outcome table and tree for this example are given below. Hopefully, the correspondence between the two is clear. The outcomes are determined by reference to the desired targeting of the vouchers. We know that if m is **true** then the customer should be sent both sets of vouchers, which completes the outcomes for the first four rows of the table. Otherwise, if g and w are both **true** then both sets of vouchers should be sent. If g is **true** and w is **false** then only the vouchers for organic products should be sent, while if g is **false** and w is **true** then only the vouchers for detergents should be sent. If all of m , w and g are **false**, then no vouchers are sent.

m	g	w	outcome
T	T	T	"All"
T	T	F	"All"
T	F	T	"All"
T	F	F	"All"
F	T	T	"All"
F	T	F	"Organics"
F	F	T	"Detergents"
F	F	F	"None"



If m is **true** then the outcome is the same whatever the truth values of g and w . We can use this fact to simplify the tree significantly, as on the right. This simplified tree corresponds to a nested conditional statement as below.



```

var vouchers in mString
if (m) then
  {vouchers.setString("All")}
else
{
  if (g) then
  {
    if (w) then
      {vouchers.setString("All")}
    else
      {vouchers.setString("Organics")}
  }
  else
  {
    if (w) then
      {vouchers.setString("Detergents")}
    else
      {vouchers.setString("None")}
  }
}
}

```

The code in Example 3.1 reflects the structure of the simplified outcome tree. The way the **if then else** statements are nested within each other reflects the branching of the tree. The code could be constructed from the

Outcome trees *are* binary trees. However, we draw outcome trees “on their side”, because the relation to the outcome table and to the code below is then more easily seen.

outcome table just as well. When analysing outcomes yourself, draw up either a table or a tree, whichever you find convenient.

It is worth noting that the order of the variables m , g , w affects the pattern of the outcomes, and hence the pattern of the corresponding code. There will be fewer branches if outcomes are grouped together. However, we will not ask you to find the simplest form.

We give below a strategy for producing nested conditional statements to achieve desired outcomes.

Strategy 3.1

Constructing conditional statements

1. Define propositions that control the desired outcome.
2. Draw up an outcome table.
3. Draw and simplify the outcome tree.
4. Use the outcome tree to construct the conditional statement.

Indeed there is sometimes a natural ordering which should be used, even if it does not give the simplest statement.

EXAMPLE 3.2

A piece of software is needed to control the traffic lights at a road junction (see Figure 3.1). There are sensors at each of the four approaches, which register when cars are waiting. There are no traffic filters. The two main traffic flows are along AC and BD . The control needs to prevent, where possible, cars waiting at a red light for more than a given time and queues building up to more than a given length.

Here, we will need two pieces of code, one for when the traffic is flowing in the AC direction and one for when it is flowing in the BD direction. We will assume that the traffic is flowing in the AC direction and just discuss and write the code for this case. (When the traffic is flowing in the BD direction, the code will be essentially the same.)

Cars should not have to wait at a red light on BD for too long. If a queue has built up in the direction AC , then the direction BD is only given a green light for a short interval. If both queues are long the usual flow times are used.

We use the following propositions.

w : Traffic on BD has been waiting too long at a red light.

q : Traffic on AC is in too long a queue.

r : Traffic on BD is in too long a queue.

The following outcomes are possible.

- | | |
|-------------|--|
| “ToBD” | Change the flow to BD . |
| “ToBDShort” | Change the flow to BD for a short period. |
| “NoChange” | The traffic lights do not change (so flow remains on AC). |

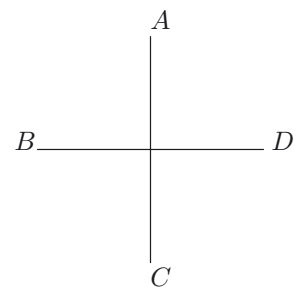


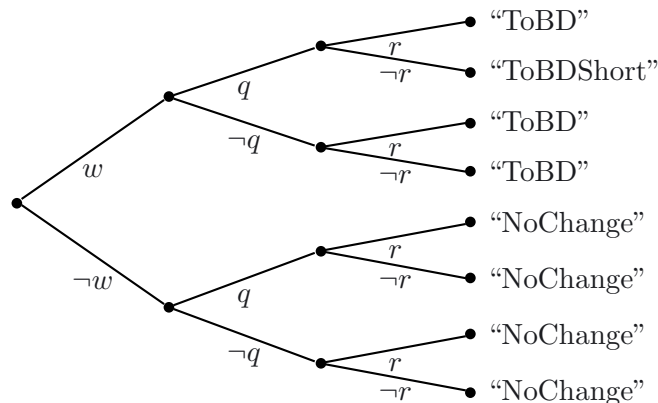
Figure 3.1

Activity 3.2

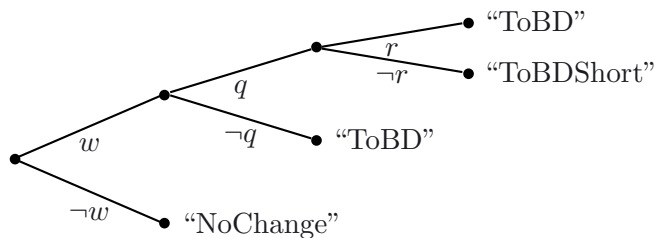
For the situation described in Example 3.2, draw up an outcome table for the propositions: w , q , r . Then construct code that assigns a suitable value to a variable *flow* of type **mString** in each case.

Discussion An outcome table and outcome tree are given below. The lights will change only when traffic on *BD* has been waiting for too long at a red light (*w* is **true**). Then there will be a change for a short period if *q* is **true** and *r* is **false**, but otherwise a change for a standard period.

<i>w</i>	<i>q</i>	<i>r</i>	outcome
T	T	T	"ToBD"
T	T	F	"ToBDShort"
T	F	T	"ToBD"
T	F	F	"ToBD"
F	T	T	"NoChange"
F	T	F	"NoChange"
F	F	T	"NoChange"
F	F	F	"NoChange"



The outcomes for $\neg w$ are all the same, so the bottom half of the tree simplifies. The outcomes for w and $\neg q$ are the same, so the top part can be simplified as well.



This tree corresponds to a nested conditional statement as below.

```
var flow in mString
if (w) then
{
  if (q) then
  {
    if (r) then
      {flow.setString("ToBD")}
    else
      {flow.setString("ToBDShort")}
  }
  else
    {flow.setString("ToBD")}
}
else
  {flow.setString("NoChange")}
```

Rather than using nested conditional statements, an alternative approach is to use a group of **if then** statements. Such a group of statements will be more clear if the conditions leading to each outcome are **complete** and **exclusive**. By exclusive, we mean that only one **if** condition can be **true** (whatever the states of the variables involved in the conditions). By complete, we mean that at least one statement will always be executed. If the grouped statements are both complete and exclusive then exactly one outcome will be executed.

Use of an outcome table can assist with ensuring that conditions are complete and exclusive. For example, the code below has the same effect as that in the discussion of Activity 3.2. It again uses the outcome tree there, but in a different way.

```
var flow in mString
if (NOT(w)) then
    {flow.setString("NoChange")}
if (w /\ NOT(q)) then
    {flow.setString("ToBD")}
if (w /\ q /\ NOT(r)) then
    {flow.setString("ToBDShort")}
if (w /\ q /\ r) then
    {flow.setString("ToBD")}
```

The code below has the same effect. The conditions there are not exclusive, but Statement (3) will replace the value of *flow* set by Statement (2), to give the required effect. Such code may seem “smart”, but its effect is likely to be significantly less clear to another person who may need to maintain it at a later date.

```
var flow in mString
if (NOT(w)) then
    {flow.setString("NoChange")}           //(1)
else
    {flow.setString("ToBD")}               //(2)
if (w /\ q /\ NOT(r)) then
    {flow.setString("ToBDShort")}         //(3)
```

Many programming languages have a construct for a group of conditional statements that are exclusive. Such constructs usually will include a default clause, which will be executed if all the conditions are **false**. Inclusion of a default clause ensures completeness.

Such constructs may have the keyword **switch** or **case**.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Draw up an outcome table from a specification.
- ☐ Given an outcome table, construct and simplify an outcome tree.
- ☐ Given an outcome tree, write code using a corresponding nested conditional statement.

Exercises on Section 3

Exercise 3.1

For Example 3.1, construct the outcome table and simplified outcome tree for the supermarket’s voucher distribution with the variables in the order w, m, g . Draw up the corresponding nested conditional statement.

Exercise 3.2

A coach company has various price bands for its tickets. There are fare reductions corresponding to the following situations.

c: The passenger has a coach card.

q: The journey is at an off-peak time.

s: There is a special offer for the journey.

There are four types of fare: standard, reduced, special and extra special. The rules relating to the type of fare are as follows.

1. A passenger can get a reduced fare if they either have a coach card or their journey is at an off-peak time.
2. A passenger without a coach card cannot have a reduction for both a special offer and an off-peak journey. The special offer always takes precedence.
3. A passenger with a coach card always has a reduced fare but cannot get an off-peak reduction as well.
4. A passenger with a coach card gets an extra special reduction for a journey with a special offer.

Use Strategy 3.1 to draw up a nested conditional statement for this situation. The code should set a variable *fare* of type **mString** to one of the values: “standard”, “reduced”, “special” or “extra special”, as appropriate.

4 Logic and language

This section is about the relationship between ordinary language and formal expressions. There is a conflict between the two, in that ordinary language is often ambiguous, whereas formal expressions must be precise and unambiguous. An important skill is the transferring of ideas from ordinary language to formal expressions and “translating” formal expressions into clear, precise language. This is particularly true when drawing up **statements of requirements** since these are usually written in ordinary language.

In this section we shall be concerned with propositions; that is, statements expressed in English which may be **true** or **false**. Propositions can be combined using $\vee$ (read “or”, as usual) or $\wedge$ (read “and”), or modified using $\neg$, so that, whatever *p* means, $\neg p$ means “not *p*”. We shall look at the combination of propositions using various operations in Subsection 4.2. First, we enrich our formal language by adding two new operations.

In this course, by “ordinary language” we will mean “English”. Similar comments apply to the relation between formal propositions and other natural languages.

4.1 Two new operations

The first new operation is denoted by $\Rightarrow$, which can be read as “implies”. It is defined using the following truth table.

a	b	$a \Rightarrow b$
T	T	T
T	F	F
F	T	T
F	F	T

The proposition $a \Rightarrow b$ roughly corresponds to the English form “if a then b ”. However, this interpretation needs to be used with caution. In natural language we might regard an assertion of this form as having no meaning when a is false, and yet the truth table gives the truth value in the last two lines as **true**. Perhaps the best way to become accustomed to this is to look at some examples.

EXAMPLE 4.1

Given a number x , let a and b be the propositions below.

a : x is a number with $x > 3$.

b : x is a number with $x^2 > 3$.

Consider the proposition $a \Rightarrow b$.

We can write this in English as

If a number is greater than three, then its square is greater than three. (A)

Expressed in English, we recognise this as a true statement.

What happens if we specify x ?

Suppose that x is 4. Then both a and b are **true**. ($4 > 3$ and $16 > 3$ are both **true**.) This fits with the first line of the truth table for $\Rightarrow$.

Now suppose that x is 1. Both a and b are then **false**. ($1 > 3$ is **false**.) This fits with the last line of the truth table.

Finally, suppose that x is 2. Then a is **false** but b is **true**. ($2 > 3$ is **false** and $4 > 3$ is **true**.) This fits with the third line of the table.

There is no value of x that fits with the second line of the truth table, which is the only line that would make the proposition $a \Rightarrow b$ **false**. This makes sense, since we regard the assertion (A) (corresponding to $a \Rightarrow b$ with a and b defined as above) as being true.

The operation $\Rightarrow$ is related to the programming structure **if then** but should not be confused with it. The purpose of the programming structure is to cause an action when the **if** condition is true, whereas $\Rightarrow$ is a binary operation on propositions; that is, a method of combining two propositions to yield a new proposition.

Activity 4.1

Given a word $aWord$ and a dictionary in alphabetical order containing the words “electricity” and “magnetism”, let e and m be the propositions below.

e : $aWord$ comes before “electricity” in the dictionary.

m : $aWord$ comes before “magnetism” in the dictionary.

Express the proposition $e \Rightarrow m$ in English, and decide whether the resulting assertion is true or false. Give three possible values of $aWord$ for which the truth values of e and m illustrate three rows of the truth table. Can you find a value of $aWord$ that illustrates the other row?

Discussion We can translate $e \Rightarrow m$ as:

If a word comes before “electricity” in the dictionary then it comes before “magnetism” in the dictionary.

This assertion is true.

Three possible values of $aWord$ are “current”, “joule” and “volt”, which come respectively: before “electricity”; between “electricity” and “magnetism”; and after “magnetism”.

For “current”, e and m are both **true**, illustrating the first row of the table. For “joule”, e is **false** and m is **true**, illustrating the third row of the table. For “volt”, e and m are both **false**, illustrating the fourth row of the table.

We cannot find a word representing the second line of the table since that would make $e \Rightarrow m$ **false**.

The second new operation is denoted by $\Leftrightarrow$, which can be read as “if and only if”. It has the following truth table.

a	b	$a \Leftrightarrow b$
T	T	T
T	F	F
F	T	F
F	F	T

A simple way of describing this truth table is to say that $a \Leftrightarrow b$ is **true** precisely when a and b have the same truth values. It is worth noting that $a \Leftrightarrow b$ is equivalent to $(a \Rightarrow b) \wedge (b \Rightarrow a)$.

See Exercise 4.2.

EXAMPLE 4.2

Suppose that $tNow$ and $wNext$ are the propositions below.

$tNow$: Today is Tuesday.

$wNext$: Tomorrow is Wednesday.

In English we can write $tNow \Leftrightarrow wNext$ as follows.

Today is Tuesday if and only if tomorrow is Wednesday.

This assertion is true. If today is Tuesday then the propositions $tNow$ and $wNext$ are both **true**, while if today is Monday, say, then $tNow$ and $wNext$ are both **false**. This fits with the first and fourth lines of the truth table for $\Leftrightarrow$. It is not possible for one of $tNow$ and $wNext$ to be **true** and the other to be **false**.

Activity 4.2

Which of the following propositions is **true** (whatever integer value x may have)? (Remember that $2^2 = 4$ and that $(-2)^2 = 4$.)

- (a) $(x^2 == 4) \Leftrightarrow (x == 2)$.
- (b) $(x^2 == 4) \Rightarrow (x == 2)$.
- (c) $(x == 2) \Rightarrow (x^2 == 4)$.

For each of these propositions, give values of x that illustrate as many different rows in the truth table as possible.

Discussion (a) This proposition is not always **true**. If x is 2, then it is **true** $\Leftrightarrow$ **true**, which is **true**. If x is 1, say, then it is **false** $\Leftrightarrow$ **false**, which is **true**. But if x is -2 , then it is **true** $\Leftrightarrow$ **false**, which is **false**.

(b) This proposition is not always **true**. If x is 2, then it is **true** $\Rightarrow$ **true**, which is **true**. If x is 1, say, then it is **false** $\Rightarrow$ **false**, which is **true**. But if x is -2 , then it is **true** $\Rightarrow$ **false**, which is **false**.

(c) This proposition is always **true**, whatever integer x may be. If x is 2, then it is **true** $\Rightarrow$ **true**, which is **true**. If x is -2 , then it is **false** $\Rightarrow$ **true**, which is **true**. If x is any other integer, it is **false** $\Rightarrow$ **false**, which is **true**.

4.2 Composite propositions

Suppose we have some simple propositions whose interpretation in English has been given. We can use operations such as $\vee$ or $\Rightarrow$ to combine these to form new propositions. We refer to such a combination as a **composite proposition**. In Subsection 2.3, we talked about well-formed formulae. We can use the same approach to describe how composite propositions can be formed. Suppose that a and b are either simple propositions, or correctly formed composite propositions. Then new composite propositions can be constructed in any of the following ways.

- $\neg a$
- $(a \vee b)$
- $(a \wedge b)$
- $(a \Rightarrow b)$
- $(a \Leftrightarrow b)$

We shall give $\neg$ precedence when interpreting composite propositions, as we did in Boolean expressions. So we do not need to put brackets around $\neg a$ in larger composite propositions.

In the next two activities we shall practise determining the truth values of composite propositions.

Activity 4.3

Consider the following propositions about words in a very small dictionary, $Dict = \{\text{"aardvark"}, \text{"aardvogel"}, \text{"qanat"}, \text{"qintar"}, \text{"quadrans"}, \text{"syzygy"}, \text{"zymurgy"}\}$.

f : The first letter of "aardvark" is 'a'.

u : There is no 'u' in "qanat".

l : "aardvogel" is 8 letters long.

y : There are 3 'y's in "syzygy".

For f , u , l and y as above, determine the truth value of $(f \wedge u) \vee \neg(l \vee y)$.

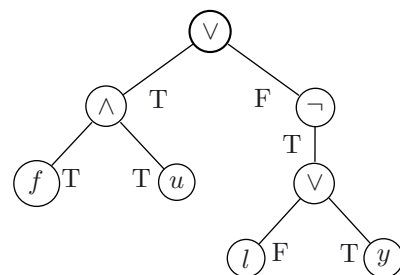
Discussion We first note that f, u and y are **true** and that l is **false**.

Thus the truth value of the given composite proposition is $(\text{true} \wedge \text{true}) \vee \neg(\text{false} \vee \text{true})$.

Now $\text{true} \wedge \text{true} = \text{true}$ and $\text{false} \vee \text{true} = \text{true}$.

Thus the given proposition has truth value $\text{true} \vee (\neg \text{true})$, that is $\text{true} \vee \text{false}$, which is **true**.

You may find the parse tree (shown on the right) helpful.



Activity 4.4

The propositions a , b , c and d below concern a small set of types of bird.

$Birds = \{\text{goldcrest, sparrow, blackbird, hobby, buzzard, eagle}\}$

We have listed these birds in increasing order of size. A hobby is a type of falcon that eats dragonflies (and small birds).

a : Blackbirds are bigger than sparrows.

b : Blackbirds are bigger than goldcrests.

c : Goldcrests are bigger than buzzards.

d : Hobbies eat dragonflies.

Determine the truth value of the composite proposition $(a \Rightarrow b) \wedge (c \Rightarrow d)$.

Discussion First, determine the truth values of the simple propositions. Here, a , b , and d are **true** and c is **false**. (We determine truth values for the simple propositions from knowledge of the context they relate to. In general, you will not be expected to supply this. In this case the necessary information was supplied in the activity statement.)

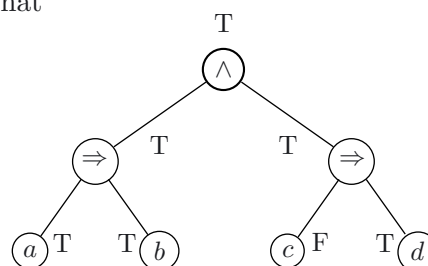
The composite proposition has the form $p \wedge q$ where p is $a \Rightarrow b$ and q is $c \Rightarrow d$. Referring to the truth table for $\Rightarrow$ on page 29, we see that

$a \Rightarrow b$ is **true** $\Rightarrow$ **true**, which is **true**, and

$c \Rightarrow d$ is **false** $\Rightarrow$ **true**, which is **true**.

So the given composite proposition is **true** $\wedge$ **true**, which is **true**.

The parse tree for the composite proposition is shown on the right. Note that we can put the operation $\Rightarrow$ at a node just as we do with the operations $\wedge$ and $\vee$.



Activity 4.5

The meanings of some simple propositions are given below.

$mNow$: Today is Monday.

$mNext$: Tomorrow is Monday.

$tNow$: Today is Tuesday.

$tNext$: Tomorrow is Tuesday.

$wNow$: Today is Wednesday.

$wNext$: Tomorrow is Wednesday.

$hNow$: Today is Thursday.

$hNext$: Tomorrow is Thursday.

$fNow$: Today is Friday.

$fNext$: Tomorrow is Friday.

- Express the meaning of each of the following composite propositions in English, and give its truth value.
 - $wNow \Rightarrow hNext$.
 - $(mNow \vee tNow) \Rightarrow (tNext \vee wNext)$.
 - $(mNow \wedge fNext) \vee (wNow \wedge tNext)$.
- Using the simple propositions above and suitable operations combining them, express each of the following formally as a composite proposition.
 - Today is Thursday if and only if tomorrow is Friday.
 - If tomorrow is Thursday or Friday, then today must be Monday.
 - Today is a weekday.
- Express each of the following composite propositions in English.
 - $\neg(tNow \vee wNow)$;
 - $\neg tNow \wedge \neg wNow$.

Discussion

- (a) We first express the proposition in English, then give its truth value.
- (i) If it is Wednesday today then it is Thursday tomorrow. This is **true**.
 - (ii) If today is Monday or Tuesday, then tomorrow is Tuesday or Wednesday. This is **true**.
 - (iii) Either today is Monday and tomorrow is Friday, or today is Wednesday and tomorrow is Tuesday. This is **false**.
- (b) In each case, we first express the given proposition formally, then give any comments. (You may have noticed that the proposition in (i) is definitely **true**; that in (ii) is definitely **false**, while the truth value of the proposition in (iii) depends on what day of the week it is today.)
- (i) $hNow \Leftrightarrow fNext$. The operation $\Leftrightarrow$ corresponds to “if and only if”.
 - (ii) $(hNext \vee fNext) \Rightarrow mNow$. Notice how English may not express composite propositions in exactly the way we might expect. We need to treat “must be” simply as meaning “is” here.
 - (iii) $mNow \vee tNow \vee wNow \vee hNow \vee fNow$.
A weekday is either Monday, Tuesday, Wednesday, Thursday or Friday.
- (c) (i) It is not the case that today is Tuesday or Wednesday.
(ii) Today is not Tuesday and today is not Wednesday.
You may notice that the assertions expressed in English in (i) and (ii) mean the same thing. You may also realise that the Boolean expressions $\neg(p \vee q)$ and $\neg p \wedge \neg q$ are equivalent. For equivalent Boolean expressions such as these, compound propositions with the corresponding structure will translate into assertions in natural language that have equivalent meanings (though they may be expressed in different ways).

See Activity 1.3.

4.3 Rewriting case statements

When maintaining code it may be necessary to ascertain what a given piece of code involving nested conditional statements actually does. For this reason we need to be able to draw up outcome tables from code. We look at an example where the code relates to the pricing of rail tickets. There are three types of ticket allowed on some rail services: standard, saver and supersaver. The type of ticket that should be issued depends on the following propositions.

f : The journey is on a Friday.

p : The journey is at peak time.

a : The ticket is being bought at least three days in advance.

Suppose that ticket sales are controlled by the following fragment of code.

```
var issue in mString
if (NOT(f \ / p) /\ a) then
    {issue.setString("supersaver")}           //(1)
if (NOT(f \ / p) /\ NOT(a)) then
    {issue.setString("saver")}                //(2)
if (f /\ a) then
    {issue.setString("saver")}                //(3)
if (p /\ a) then
    {issue.setString("saver")}                //(4)
if ((f \ / p) /\ NOT(a)) then
    {issue.setString("standard")}            //(5)
```

We considered an example of this type at the start of Section 3.

Line numbers are given in the code for reference. As usual, the values in **mString** represent what would in practice be code routines.

To draw up an outcome table for such a code fragment, we first draw up a table giving the eight possible combinations of truth values for f , p and a . For each line in the table, we then need to determine which of the code statements (1)–(5) will be executed. That will determine the outcome in each case.

Activity 4.6

Draw up an outcome table for the code fragment above.

Discussion The outcome table is given below. In each case, we have shown which code statement in the fragment will be executed. For example, if f is **true**, and p and a are **false**, then the only **if** condition that is **true** is that controlling (5). The consequent outcome is that *issue* is set to “standard”. It is, of course, possible to write a series of **if then** statements such as this in which more than one **if** condition is **true**. In such a situation, the outcome is determined by the last **if** condition that is **true**. Here, if f , p and a all take the value **true** then the **if** conditions controlling both statements (3) and (4) are **true**. This is recorded in the table for reference. In practice, it is (4) that would determine the outcome, since that would overwrite any value stored in *issue* at (3). Here, both statements (3) and (4) lead to the same outcome, but this is still an indication that the code is not well designed. In well designed code, the **if** conditions would not overlap.

If different **if** conditions can be **true** and would lead to different outcomes, then one would need be even more concerned about the code design.

f	p	a	outcome	Code statement
T	T	T	“saver”	(3),(4)
T	T	F	“standard”	(5)
T	F	T	“saver”	(3)
T	F	F	“standard”	(5)
F	T	T	“saver”	(4)
F	T	F	“standard”	(5)
F	F	T	“supersaver”	(1)
F	F	F	“saver”	(2)

The outcome table in the preceding comment has none of the outcomes grouped together. This will lead to quite lengthy code. We can reorganise the table by giving the columns for f , p and a in a different order. In the table below, we have used the order a , f , p . This has enabled us to group the outcomes. Take a moment to satisfy yourself that the two tables are the same. For example, the seventh row in the table below has a **false**, f **false** and p **true**. That corresponds to the sixth row in the previous table.

There is no preferred order for the three propositions.

a	f	p	outcome
T	T	T	“saver”
T	T	F	“saver”
T	F	T	“saver”
T	F	F	“supersaver”
F	T	T	“standard”
F	T	F	“standard”
F	F	T	“standard”
F	F	F	“saver”

Activity 4.7

Use the second outcome table to draw up a nested conditional statement to replace the original code.

Discussion Code corresponding to the second outcome table is given below.

```
var issue in mString
if (a) then
{
  if (f) then
    {issue.setString("saver")}
  else
  {
    if (p) then
      {issue.setString("saver")}
    else
      {issue.setString("supersaver")}
  }
}
else
{
  if (f) then
    {issue.setString("standard")}
  else
  {
    if (p) then
      {issue.setString("standard")}
    else
      {issue.setString("saver")}
  }
}
```

To relate the code to the application situation, we can use the meanings of the simple propositions given earlier to supply interpretations of composite propositions relating to the code. For example, we see from the second outcome table that a supersaver is only issued if the proposition $a \wedge \neg f \wedge \neg p$ is **true**, while a saver ticket is issued if the proposition $(a \wedge (f \vee p)) \vee (\neg a \wedge \neg f \wedge \neg p)$ is **true**.

Activity 4.8

- (a) Give interpretations in English of each of the following composite propositions, where the meanings of the simple propositions a , f and p are as given at the start of this subsection.
 - (i) $a \wedge \neg f \wedge \neg p$. (ii) $(a \wedge (f \vee p)) \vee (\neg a \wedge \neg f \wedge \neg p)$.
- (b) Write down a formal proposition that is **true** when a standard ticket is issued. Give an interpretation of this in English.

Discussion

- (a) Interpretations can be expressed in various ways, but should be equivalent to the following.
- (i) The ticket is bought at least three days in advance, and the journey is not on a Friday and not at peak time.
 - (ii) Either the ticket is bought at least three days in advance and the journey is either on a Friday or at peak time, or the ticket is not bought at least three days in advance and the journey is not on Friday and not at peak time.
- (b) A suitable formal proposition is $\neg a \wedge (f \vee p)$, which can be interpreted as:
The ticket is not bought at least three days in advance, and the journey is either on Friday or at peak time.
-

With these interpretations the pricing policy makes some sense. The ticket costs more if the journey is either on Friday or at peak time. There is a reduction if you buy the ticket at least three days in advance. The original code did not make this apparent.

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Interpret the notation $\Rightarrow$ and $\Leftrightarrow$ in composite propositions.
- ☐ Translate between formal composite propositions and natural language.
- ☐ Find the truth value of composite propositions.
- ☐ Draw up an outcome table from a code fragment.

Exercises on Section 4

Exercise 4.1

Given a number x , let p , q and r be the propositions below.

p : x is a number for which $x == 10$ is **true**.

q : x is a number for which $x + 2 == 12$ is **true**.

r : x is a number for which $x > 0$ is **true**.

- (a) Write down the proposition $p \Leftrightarrow q$ in English. Decide whether it is **true** or **false**. Choose two values for x so that the truth values of p and q illustrate two rows of the truth table. Can you find a value of x that illustrates either of the other two rows?
- (b) Express in English each of the propositions: (i) $\neg p \vee r$; (ii) $\neg r \vee p$. Can you find a value of x that makes either of these propositions **false**?

Exercise 4.2

Draw up the truth tables for $\neg a \vee b$ and for $(a \Rightarrow b) \wedge (b \Rightarrow a)$. Compare the final columns of your tables with the final columns of the tables for $\Rightarrow$ and $\Leftrightarrow$. What do you notice?

Exercise 4.3

- (a) Let f , u , l and y be the propositions about *DICT* given in Activity 4.3. Find the truth value of $(f \vee u) \Rightarrow (l \wedge y)$.
- (b) For the simple propositions defined as in Activity 4.5, express in English the composite proposition $(wNow \wedge hNext) \vee (wNext \Rightarrow tNow)$. What is its truth value?

Exercise 4.4

An airline sells tickets on the web. The following propositions determine the availability and price of the ticket.

f : The aeroplane is full.

e : The aeroplane is less than a quarter full.

w : The journey is at a weekend.

The current code being used is given below.

```
var issue in mString
if (f) then
  {issue.setString("no ticket")}           //(1)
else
{
  if (NOT(f) /\ e /\ w) then
    {issue.setString("very cheap ticket")} //(2)
  else
  {
    if (NOT(f) /\ e) then
      {issue.setString("cheap ticket")}    //(3)
    else
    {
      if (NOT(e)) then
        {issue.setString("standard ticket")} //(4)
      }
    }
  }
}
```

- Draw up an outcome table for this code. Then follow Strategy 3.1 to draw up a nested conditional statement with an equivalent effect.
- There are four possible outcomes here (*issue* is set to one of four values: “no ticket”, etc). Use your outcome table to give a single proposition under which each of these four outcomes occurs. Express in English the circumstances under which the outcome is “standard ticket”.

Summary of Unit 9

- A Boolean expression consists of a number of atoms (representing Boolean variables) combined using the operations $\neg$, $\vee$, $\wedge$, $\Rightarrow$ and $\Leftrightarrow$. We can show the truth value of a Boolean expression for all possible combinations of truth values of the atoms using a truth table. The disjunctive normal form of a Boolean expression has a term corresponding to each row in the truth table that evaluates to **true**. Two Boolean expressions are equivalent if their truth values are the same for any combination of

truth values of their atoms. Equivalent Boolean expressions have identical truth tables. We use a standard order for the rows in a truth table; this makes it much easier to compare truth tables.

2. A Boolean expression with correct syntax can be represented by a parse tree. We gave rules that produce a Boolean expression that is a well-formed formula, and so is certain to have correct syntax. A well-formed formula has enough brackets to ensure that it can be interpreted, but we can reduce the number of brackets by giving precedence to negation in Boolean expressions. The key to finding the parse tree for a given expression lies in identifying the operation to place at the root of the tree. This can be done by counting brackets, so long as brackets have been included in the appropriate way.
3. For code involving conditional statements, we can draw up an outcome table. This may either be an expression of the outcomes required of the code, or a way of analysing a given code fragment. For a table of outcomes, we can give a corresponding outcome tree. If adjacent branches lead to the same outcome, then the outcome tree can be simplified. A nested conditional statement can be given with structure corresponding to an outcome tree.
4. A proposition is an assertion about some context that may be either **true** or **false**. (We use our understanding of the context to assign the appropriate truth value to a proposition.) Statements of requirements governing the design of software may be expressed using propositions. Given simple propositions, we can form composite propositions by combining them using $\neg$, $\vee$, $\wedge$, $\Rightarrow$ and $\Leftrightarrow$. The operation $\Rightarrow$ can be read as “implies” and the operation $\Leftrightarrow$ can be read as “if and only if”. With known interpretations of the simple propositions, such composite propositions correspond to more complicated statements in natural language. Translation between natural language and formal expressions is an important skill in formulating an unambiguous statement of requirements.

Solutions to the exercises

Solution 1.1

- (a) The truth tables for first $(a \vee b) \vee c$, then $a \vee (b \vee c)$ are given below. Note that their final columns are identical, showing that these expressions are equivalent.

a	b	c	1 $(a \vee b)$	2 $\vee c$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	T	T
F	T	T	T	T
F	T	F	T	T
F	F	T	F	T
F	F	F	F	F

which gives

a	b	c	$(a \vee b) \vee c$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	T
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	F

a	b	c	2 $(b \vee c)$	1 $a \vee$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	T
F	T	T	T	T
F	T	F	T	T
F	F	T	T	T
F	F	F	F	F

which gives

a	b	c	$a \vee (b \vee c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	T
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	F

- (b) The truth tables for first $(a \wedge b) \vee c$, then $a \wedge (b \vee c)$ are given below. Note that their final columns are not the same (they differ in the fifth and seventh rows), showing that these expressions are not equivalent.

a	b	c	1 $(a \wedge b)$	2 $\vee c$
T	T	T	T	T
T	T	F	T	T
T	F	T	F	T
T	F	F	F	F
F	T	T	F	T
F	T	F	F	F
F	F	T	F	T
F	F	F	F	F

which gives

a	b	c	$(a \wedge b) \vee c$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	F
F	T	T	T
F	T	F	F
F	F	T	T
F	F	F	F

a	b	c	2 $(b \vee c)$	1 $a \wedge$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	F
F	T	T	T	T
F	T	F	T	T
F	F	T	T	T
F	F	F	F	F

which gives

a	b	c	$a \wedge (b \vee c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	F
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	F

Solution 1.2

The truth tables for $a \vee (b \wedge c)$ then $(a \vee b) \wedge (a \vee c)$ are given below. Their final columns are identical, showing that these expressions are equivalent.

a	b	c	2 $\vee$	1 $(b \wedge c)$
T	T	T	T	T
T	T	F	T	F
T	F	T	T	F
T	F	F	T	F
F	T	T	F	T
F	T	F	F	F
F	F	T	F	F
F	F	F	F	F

which gives

a	b	c	$a \vee (b \wedge c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	T
F	T	T	T
F	T	F	F
F	F	T	F
F	F	F	F

a	b	c	2 $(a \vee b)$	3 $\wedge$	1 $(a \vee c)$
T	T	T	T	T	T
T	T	F	T	T	F
T	F	T	T	T	T
T	F	F	T	T	F
F	T	T	F	T	T
F	T	F	F	F	F
F	F	T	F	F	T
F	F	F	F	F	F

which gives

a	b	c	$(a \vee b) \wedge (a \vee c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	T
F	T	T	T
F	T	F	F
F	F	T	F
F	F	F	F

Solution 1.3

(a) The truth table for $a \wedge (b \vee \neg a)$ is given below.

a	b	2 $\wedge$	1 $(b \vee \neg a)$
T	T	T	F
T	F	F	F
F	T	F	T
F	F	F	T

which gives

a	b	$a \wedge (b \vee \neg a)$
T	T	T
T	F	F
F	T	F
F	F	F

In this case, the disjunctive normal form consists of the single term $a \wedge b$.

(b) The truth table for $a \vee (\neg b \wedge c)$ is given below.

a	b	c	2 $\vee$	1 $(\neg b \wedge c)$
T	T	T	T	F
T	T	F	T	F
T	F	T	T	T
T	F	F	T	F
F	T	T	F	F
F	T	F	F	F
F	F	T	F	T
F	F	F	F	F

which gives

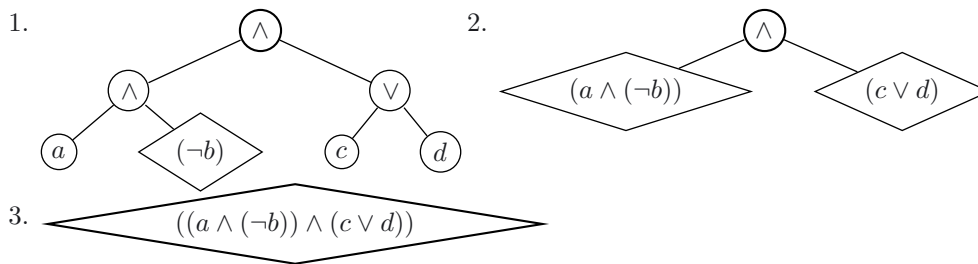
a	b	c	$a \vee (\neg b \wedge c)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	T
F	F	F	F

The disjunctive normal form is:

$$(a \wedge b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge \neg b \wedge c) \vee (a \wedge \neg b \wedge \neg c) \vee (\neg a \wedge \neg b \wedge c).$$

Solution 2.1

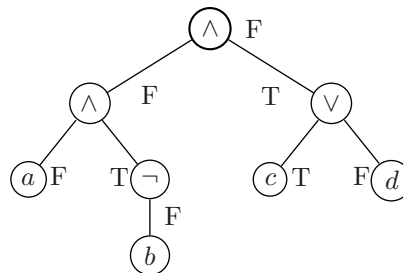
The Boolean expression is derived from the tree as follows:



So the tree in Figure 2.3 represents the expression $((a \wedge (\neg b)) \wedge (c \vee d))$. You may have used the precedence rule for $\neg$ to write it as $(a \wedge \neg b) \wedge (c \vee d)$.

To find the truth value of the expression when a , b and d are **false** and c is **true**, we can annotate the tree.

We can see from this that the required truth value is **false**.



Solution 2.2

Remember that you can use bracket counting to find the root node, if necessary. To do that, we must add brackets around the negations, and then remove the outer brackets round the entire expression, to get

$$(a \wedge (\neg b)) \vee ((\neg c) \vee d).$$

Step 1

The root node is $\vee$ with two subexpressions $(a \wedge (\neg b))$ and $((\neg c) \vee d)$.

Step 2

The expression $(a \wedge (\neg b))$ has $\wedge$ as root with subexpressions a and $(\neg b)$.

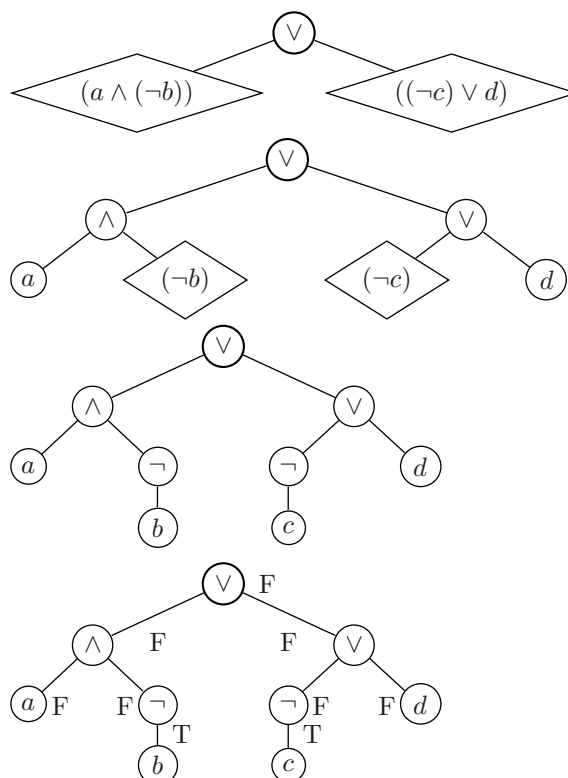
The expression $((\neg c) \vee d)$ has $\vee$ as a root node and has subexpressions $(\neg c)$ and d .

Step 3

We replace $(\neg b)$ by a root node $\neg$ and a subexpression b . Similarly, replace $(\neg c)$ by a root node $\neg$ and a subexpression c . This gives us the parse tree for the given expression.

The truth value of the expression when b and c are **true** and a and d are **false** can be obtained by annotating the tree.

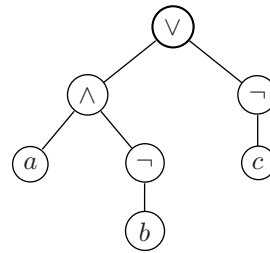
We can see from this that the required truth value is **false**.



Solution 2.3

A parse table and parse tree for the expression $(a \wedge (\neg b)) \vee (\neg c)$ are given below.

Number	Formula	How formed
1	b	atom
2	a	atom
3	$(\neg b)$	negation(1)
4	c	atom
5	$(a \wedge (\neg b))$	conjunction(2,3)
6	$(\neg c)$	negation(4)
7	$((a \wedge (\neg b)) \vee (\neg c))$	disjunction(5,6)



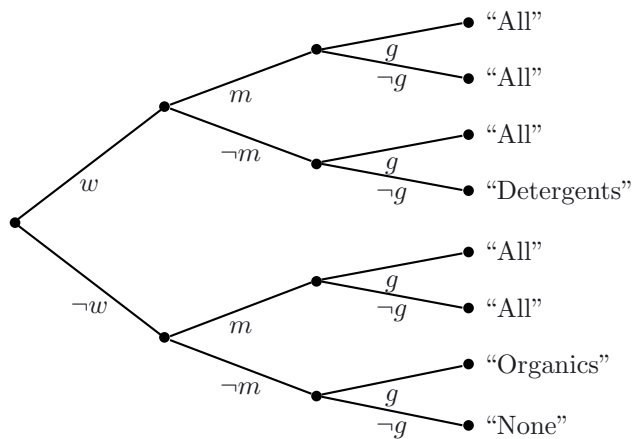
To draw the parse tree, we do not need to add or to remove any brackets. The fact that we can draw a parse tree shows that the expression is syntactically correct. The fact that we need not add any brackets shows that the precedence rule for negation is not required to interpret it. The fact that we did not need to remove outer brackets shows that the given expression is not precisely in the form of a well-formed formula, which would have an additional pair of outer brackets, as in $((a \wedge (\neg b)) \vee (\neg c))$.

Solution 3.1

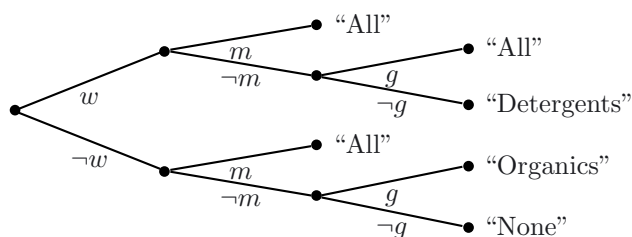
Draw up an outcome table with the first three columns in the order: w , m , g , and then rows giving the possible truth values in the standard order, as shown to the right. The appropriate outcomes can be obtained by matching each row with the outcome table in Example 3.1. For example, the third row here (with w **true**, m **false** and g **true**) corresponds to the fifth row of the table in Example 3.1.

w	m	g	outcome
T	T	T	"All"
T	T	F	"All"
T	F	T	"All"
T	F	F	"Detergents"
F	T	T	"All"
F	T	F	"All"
F	F	T	"Organics"
F	F	F	"None"

The outcome tree is then



This simplifies to



The associated code is given below.

```

var vouchers in mString
if (w) then
{
  if (m) then
    {vouchers.setString("All")}
  else
  {
    if (g) then
      {vouchers.setString("All")}
    else
      {vouchers.setString("Detergents")}
  }
}
else
{
  if (m) then
    {vouchers.setString("All")}
  else
  {
    if (g) then
      {vouchers.setString("Organics")}
    else
      {vouchers.setString("None")}
  }
}
}

```

Solution 3.2

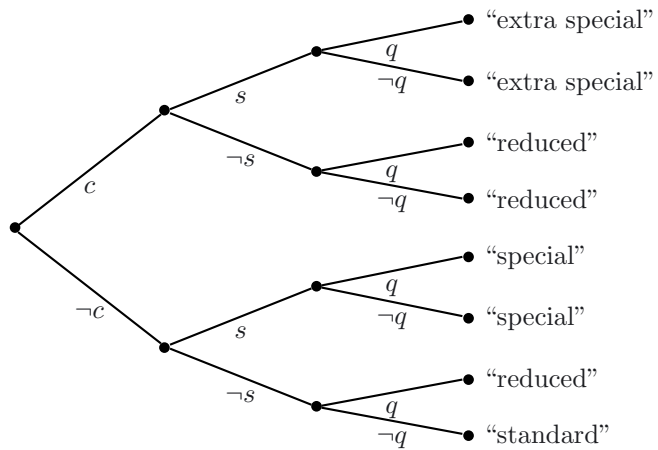
The outcome table below has the variables in the order given in the question. The outcomes in rows 1 and 3 come from Rule 4 given in the question. The outcomes in rows 2, 4 and 6 come from Rule 1 (row 2 also uses Rule 3). The outcome in row 5 comes from Rule 2. In row 8, none of the fare reductions applies, so it is reasonable to assume that the standard fare is then used.

<i>c</i>	<i>q</i>	<i>s</i>	outcome
T	T	T	“extra special”
T	T	F	“reduced”
T	F	T	“extra special”
T	F	F	“reduced”
F	T	T	“special”
F	T	F	“reduced”
F	F	T	“special”
F	F	F	“standard”

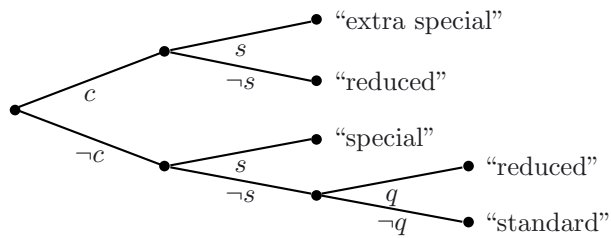
The pattern of this table suggests that exchanging the order of the columns to *c*, *s*, *q*, will provide some grouping of outcomes. (You may not have done this; do not worry if you wrote code based on the original outcome table.)

<i>c</i>	<i>s</i>	<i>q</i>	outcome
T	T	T	“extra special”
T	T	F	“extra special”
T	F	T	“reduced”
T	F	F	“reduced”
F	T	T	“special”
F	T	F	“special”
F	F	T	“reduced”
F	F	F	“standard”

Using the second table, the outcome tree is



This simplifies to



The associated code is given below.

```
var fare in mString
if (c) then
{
  if (s) then
    {fare.setString("extra special")}
  else
    {fare.setString("reduced")}
}
else
{
  if (s) then
    {fare.setString("special")}
  else
  {
    if (q) then
      {fare.setString("reduced")}
    else
      {fare.setString("standard")}
  }
}
```

Solution 4.1

(a) $p \Leftrightarrow q$ can be expressed in English as below.

A number is equal to ten if and only if adding two to it gives a number equal to twelve.

This proposition is **true**.

If x is 10 then both p and q are **true**, which illustrates the first row of the truth table.

If x is any other number, such as 5, then both p and q are **false**, which illustrates the last row of the truth table.

We cannot find values illustrating the other two rows of the truth table.

(b) These propositions can be expressed in English as follows.

(i) A number is either not equal to 10, or it is greater than 0.

(ii) A number is either not greater than 0 or it is equal to 10.

We cannot find a number x that makes (i) **false**.

We can find numbers that make (ii) **false**: for example, try 5 for x .

Solution 4.2

a	b	$\neg a$	$\vee$	b
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	F

This is the same as the truth table for $a \Rightarrow b$. Thus $\neg a \vee b$ and $a \Rightarrow b$ are equivalent.

For the next truth table, note that $b \Rightarrow a$ is **false** only when b is **true** and a is **false**.

a	b	$a \Rightarrow b$	$\wedge$	$b \Rightarrow a$
T	T	T	T	T
T	F	F	F	T
F	T	T	F	F
F	F	T	T	T

So the truth table for $(a \Rightarrow b) \wedge (b \Rightarrow a)$ is the same as that for $a \Leftrightarrow b$.

Thus $(a \Rightarrow b) \wedge (b \Rightarrow a)$ and $a \Leftrightarrow b$ are equivalent.

Solution 4.3

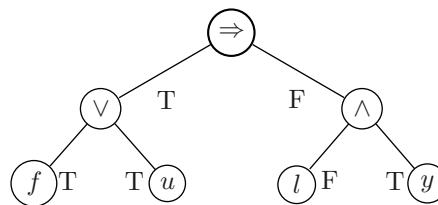
(a) We know that f , u and y are **true** and l is **false**.

So $f \vee u$ is **true**, while $l \wedge y$ is **false**.

Then the given proposition is

true $\Rightarrow$ **false**, and so is **false**.

An annotated parse tree is given to the right.



(b) We can express this proposition in English as follows.

Either today is Wednesday and tomorrow is Thursday, or if tomorrow is Wednesday then today is Tuesday.

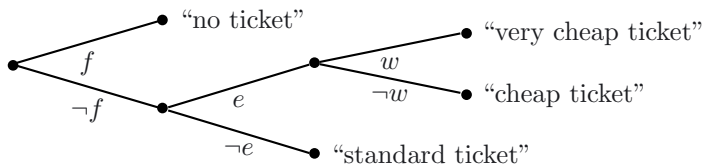
We do not know the truth value of $wNow \wedge hNext$, since we do not know what day it is today. However, we can be sure that $wNext \Rightarrow tNow$ is **true**. So the given proposition must be **true**. A proposition of the form $p \vee q$ will be **true** if either p or q is **true**, regardless of the truth value of the other part.

Solution 4.4

(a) The outcome table is given below.

f	e	w	outcome	code statement
T	T	T	"no ticket"	(1)
T	T	F	"no ticket"	(1)
T	F	T	"no ticket"	(1)
T	F	F	"no ticket"	(1)
F	T	T	"very cheap ticket"	(2)
F	T	F	"cheap ticket"	(3)
F	F	T	"standard ticket"	(4)
F	F	F	"standard ticket"	(4)

We go directly to a simplified outcome tree.



Code corresponding to the simplified outcome tree is given below.

```

var issue in mString
if (f) then
  {issue.setString("no ticket")}
else
  {
    if (e) then
    {
      if (w) then
        {issue.setString("very cheap ticket")}
      else
        {issue.setString("cheap ticket")}
    }
    else
      {issue.setString("standard ticket")}
  }
}

```

(b) Suitable propositions are tabulated below.

proposition	outcome
f	"no ticket"
$\neg f \wedge e \wedge w$	"very cheap ticket"
$\neg f \wedge e \wedge \neg w$	"cheap ticket"
$\neg f \wedge \neg e$	"standard ticket"

The outcome "standard ticket" corresponds to the formal proposition $\neg f \wedge \neg e$, that is:

The aeroplane is not full and is not less than a quarter full.

Index

- $\neg$, 5
- $\Leftrightarrow$, 30
- $\Rightarrow$, 29
- atom, 5
- Boolean expression, 4
- Boolean search, 5
- Boolean variable, 5
- bracket counting, 18
- complete sum-of-products form, 10
- composite proposition, 31
- disjunctive normal form, 10
- equivalent (Boolean expressions), 8
- group of conditional statements
 - complete, 26
 - exclusive, 26
- nested conditional statement, 24
- outcome table, 23
- outcome tree, 23
- parse table, 16
- parse tree, 11
- precedence of negation, 7
- proposition, 21
 - composite, 31
 - simple, 31
- statement of requirements, 28
- subexpression, 18
- syntax, 15
- truth table, 5
- well-formed formula, 15

